Measure AI by Motion — GTM AI Strategy & Technical Plan of Record

Oct 12, 2026 · @Andy P

1. Executive strategic narrative

Fund motions, not heads. Every dollar of GTM AI spend should carry the Jira ticket of the motion it serves, and be joined in Databricks to the Salesforce campaign that motion exists to move. Do that, and cost control and ROI stop being two worries: they become the same number. Everything else in this plan (the context architecture, the governed harness, the role dashboards and the GTM engineering workflows) exists to make that number cheaper to produce, more accurate, or visible to the people who act on it.

> AI is the lever, the motion is the unit, the campaign is where cost and return meet.

The problem we are actually solving

The current plan is role-based dashboards plus per-person spend caps on the company-wide Databricks AI gateway. The intent is right: put Claude Code in every GTM user's hands without runaway cost. But the gateway today shows spend per model and nothing else. There is no per-user, per-tool or per-purpose view, and no way to tie a single dollar to a campaign. The gateway sees spend; it cannot see intent.

A per-person cap is the control you reach for when you are blind, and it fails in predictable ways:

- **It punishes the best users.** The people doing the most exploratory, valuable work hit the limit first and are throttled when most productive.
- **It is blind to value.** A dollar on a pipeline-generating campaign and a dollar on idle noodling look identical.
- **It funds winners and losers equally.** Every head gets the same safe slice, regardless of return.
- **It drives usage underground.** People ration, hide usage, or route work to tools outside the gateway, which makes the data worse.
- **It misreads low usage as good.** Someone using no AI looks like a model citizen but may be doing the work by hand at far higher cost.

The underlying error is treating AI spend as a cost to contain per head instead of an investment to allocate per outcome. The fix is sight, not a smaller budget.

The reframe: the motion is the unit

A **motion** is a real GTM workflow with a business object attached: turning a trade-show list into a running campaign, briefing an AE on an account, resolving an event list, scoring accounts against the ICP. Each motion is declared as a Jira ticket, and the ticket links to a Salesforce campaign, its opportunities and their accounts.

The shift is in the organizing principle, not the data. The gateway already records usage; we change what it is grouped by. Motion-first measurement is strictly more powerful than role-first: every ticket has an owner, so role views fall out of the motion model for free. Built the other way, motion views are impossible.

	Today: cap spend per head	Proposed: fund spend per motion
Answers	Who spent?	What did it produce?
Unit of control	The individual	The motion (Jira ticket)
Budget	Same slice for every head	Flows to motions that return
High-return work	Throttled first	Allowed to scale
Low AI usage	Misread as efficiency	A question: manual work, or a tool we can't see
Control	Limits	Visibility into value

How every workstream follows from the thesis

The five bodies of work in this document are not parallel initiatives. Each one serves the motion ledger in a specific way, and each is sequenced by when the ledger needs it.

Workstream	Its job under the motion thesis	What it gives the ledger
Motion ledger (section 3)	Defines the unit and computes the number	Cost per motion, cost avoided, goal attainment, influenced pipeline
Context architecture (section 4)	Spend less by knowing more: preload interpretive context, make the campaign node the shared anchor	Fewer model steps and tool calls per motion; the campaign crosswalk is the join path
Governed harness and gateway (section 5)	Every call carries its motion stamp and stays on a governed path	Request tags, hooks that refuse untagged work, a complete event log
Role dashboards and lead card (section 6)	Put each motion's outputs and costs in front of the role that acts on them	The read surface; role is a filter over motions
GTM engineering workflows (section 7)	The motions themselves	The proof motion and the next motions to fund

What changes in the current plan, and what stays

- **Stays:** the gateway's org-level hard cap (as a loose ceiling, not the control model), role-based views, and a team plugin that ships rules and skills.
- **Changes:** per-person caps give way to per-motion budgets taken from each campaign's own budget. Role-level daily budgets in the harness become a transitional guardrail, retired as motion visibility comes online.
- **Resequenced:** role dashboards, the shared query cache and the full knowledge wiki are built after the ledger exists, and each must show its effect on cost per motion to earn the next phase.

The pitch to leadership is not "your plan is wrong". It is "this is how we get what your plan is trying to achieve": a targeted control instead of a blunt one.

The proof and the ask

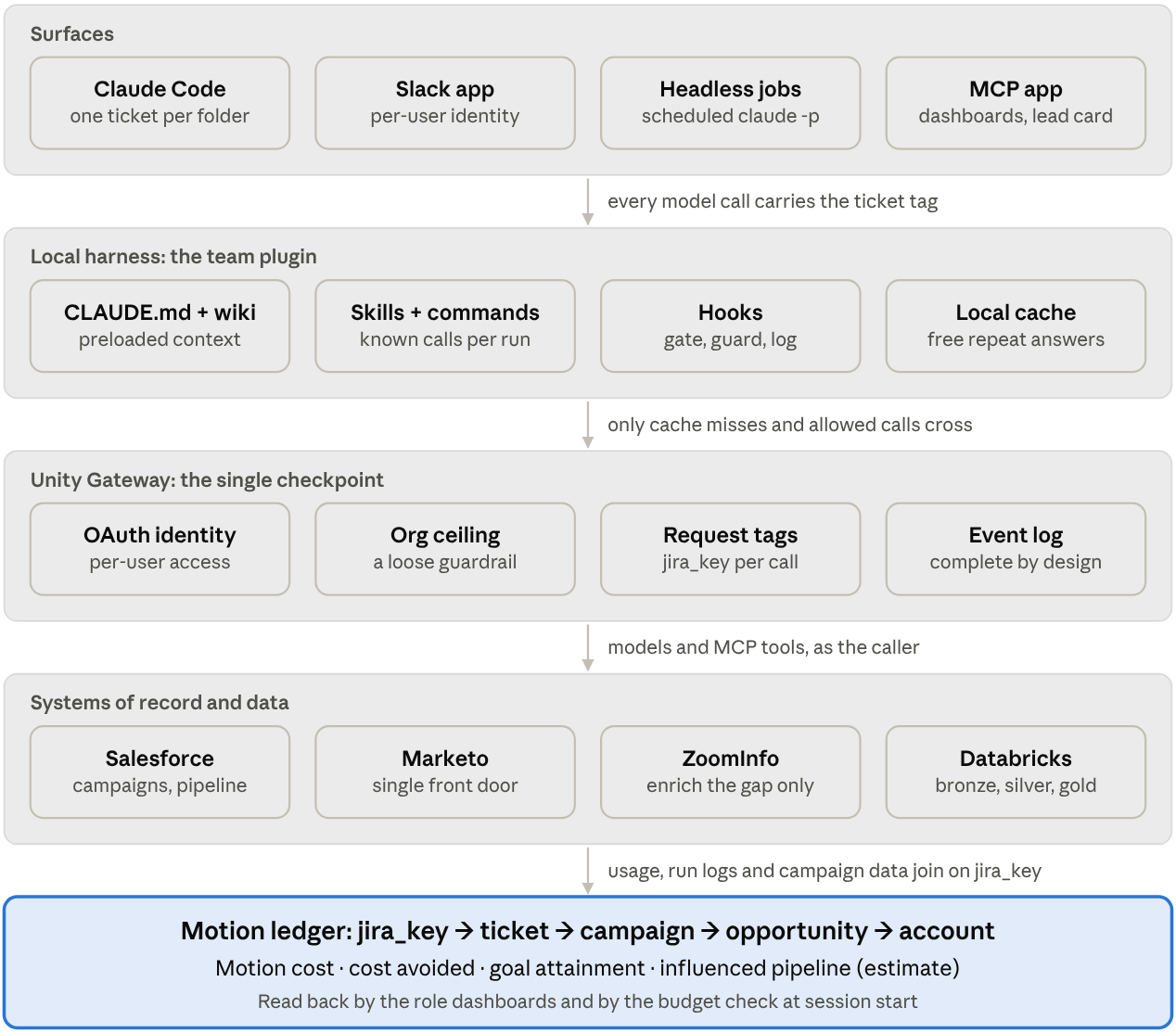
The proof is one real motion rebuilt end to end: a trade-show list turned into a running campaign. It checks what we already own in Databricks before paying ZoomInfo to enrich, loads clean into Marketo, ties to a Salesforce campaign with an owner, budget, dates and goal, and makes every token traceable to that campaign.

The ask: about two weeks to build that proof on a real list; read access (or a filtered view) on the gateway usage table, which is admin-only by default; and a nod to build one example before committing to role-based dashboards. Then decide together whether to scale.

What success looks like

Each motion reports three honest numbers: **total motion cost** from the ledger, **goal attainment** against the campaign's own pre-set goal, and **influenced pipeline**, always labeled an estimate. Beside them sits **cost avoided**: ZoomInfo credits not spent because records matched internally. Leadership can then fund the motions that return, fix or stop the ones that don't, and point AI at the motions still done by hand.

Every layer feeds one motion ledger, keyed by the Jira ticket



system at a glance · 4 layers feeding 1 ledger

Each layer has one job in the motion model: surfaces declare the motion, the harness keeps work on a governed path, the gateway meters it under the caller's identity, and the ledger turns it into cost and return per motion.

2. Operating principles and reconciled decisions

Twelve principles govern every design choice below, and each one serves the first. Where the six source documents disagreed, the motion thesis decides; the reconciliation table records each call.

Principles

1. **Measure by motion, not by tool or role.** A motion is a workflow with a business object attached. Role, person, tool and model are dimensions of it.
2. **One stamp.** Every model call and every pipeline run carries exactly one key, the Jira ticket. Campaign, opportunity, account, owner and team are reached by joins at query time: one stamp, infinite cuts.
3. **The ID rides around the model, not through it.** The key travels as request metadata, never prompt text, so the model cannot mangle it and truncation cannot drop it. Bulk records stay in Databricks and are processed with set-based SQL; the model orchestrates.
4. **Tie every dollar to an outcome.** Every campaign carries an owner, a budget, start and end dates, and a goal, set before the first touch.
5. **Check what we own before we pay to re-learn it.** Databricks before ZoomInfo; preloaded context before tool calls; cache before a fresh query.
6. **One choke point.** The gateway is the only path to models and MCP tools, so its log is complete by construction. Whatever routes around it is mapped and declared as outside the measurement boundary.
7. **Restrict downward, never loosen.** Org ceiling, then team, then user. Deny beats allow at every tier; the effective budget is the minimum of the tiers.
8. **Control is visibility, not limits.** Keep a loose guardrail during transition, start every gate in warn mode, and retire hard caps as motion-level visibility comes online.
9. **Judgment in CLAUDE.md, must-happen work in code.** CLAUDE.md is guidance; hooks, skills and settings are enforcement. Logic lives in versioned skills, not vendor UIs.
10. **Identity decides access, never the model.** Per-user OAuth everywhere. Role and rows come from the signed-in identity; anything shared must be permission-free by construction.
11. **Never fabricate; label estimates.** An honest "unresolved" beats an invented email. Influenced pipeline is always labeled an estimate. Predictive connections are flagged as conjecture.
12. **Prove one before scaling.** One motion, one role, one gate at a time. Enforcement and new layers arrive only after the data earns them.

Reconciliation of the source documents

Topic	Where the sources differ	Plan of record
Budget control	The Claude Code governance design gates tools on per-role daily budgets keyed to the user. The motion framework retires per-person caps.	The budget gate is re-keyed to the motion: it reads the ticket's campaign budget. Role and user budgets survive only as a loose, warn-mode guardrail under the org ceiling, and are retired on evidence.
Join key	The motion framework stamps the Jira key. The context architecture and dashboards join on the canonical campaign ID.	The Jira key is the only stamp on AI work. The canonical campaign ID is the business key the ticket resolves to through the campaign crosswalk. Gold tables carry `canonical_campaign_id`.
How the tag is attached	An earlier plan put tagging in a hook.	Hooks act on tool calls and cannot add headers to model requests. The tag is a setting (`ANTHROPIC_CUSTOM_HEADERS` in the project's `.claude/settings.json`); hooks refuse untagged work.
Genie cost	The harness deck treats Genie as running on "its own credits".	Genie Agents usage by people is free until January 31, 2027; service principals pay from the first DBU; SQL warehouse compute is always billed. Genie is for human exploration and new cuts, never the production path, which uses deterministic SQL or Unity Catalog functions.
Live source access in the MCP app	The lead-card call path has the MCP server holding Salesforce, Marketo and Databricks credentials and running live SOQL. The dashboards design reads only Databricks gold with a read-only service principal.	v1 follows the dashboards design: gold tables only, no source-system credentials on the server. A live single-record lookup is a v2 option through Salesforce Hosted MCP under the user's OAuth, as a pre-declared call.

Topic	Where the sources differ	Plan of record
Gateway in the MCP app path	The lead-card design puts the gateway in front of every `tools/call`. The dashboards design leaves it as an open question.	Route app tool calls through the gateway (as an MCP Service) so the event log is complete and calls are attributable; confirm with the gateway owner.
Wiki and semantic cache	The motion framework parks the knowledge wiki, org dashboards and semantic caching. The context architecture and harness designs build them.	Sequenced, not dropped. The proof needs only a soul file, one project file and the campaign crosswalk. The shared cache is built only if the ledger shows a meaningful share of spend on near-duplicate questions.
Hook failure mode	The motion framework sets the ticket hook to fail closed. The governance design says fail closed for writes, open for reads.	Both: write tools (ZoomInfo, Marketo, Salesforce writes) fail closed; read tools fail open, with the failure logged.
Names	Older names appear in some documents.	AI Gateway is now Unity Gateway (Unity AI Gateway); Genie spaces are now Genie Agents.

3. The Measure-by-Motion framework

Measurement happens in one derived Databricks table, the **motion ledger**, which joins gateway usage and pipeline run logs to business objects through one key: the Jira ticket. Databricks is the only place that sees both halves: AI usage on one side, campaign and revenue data on the other.

3.1 What counts as a motion

A motion is a repeatable GTM workflow with a business object attached and a Jira ticket that declares it. The ticket carries a `motion_type`, an owner and links to a Salesforce campaign or opportunity. The first motion types come straight from the workflows in this plan:

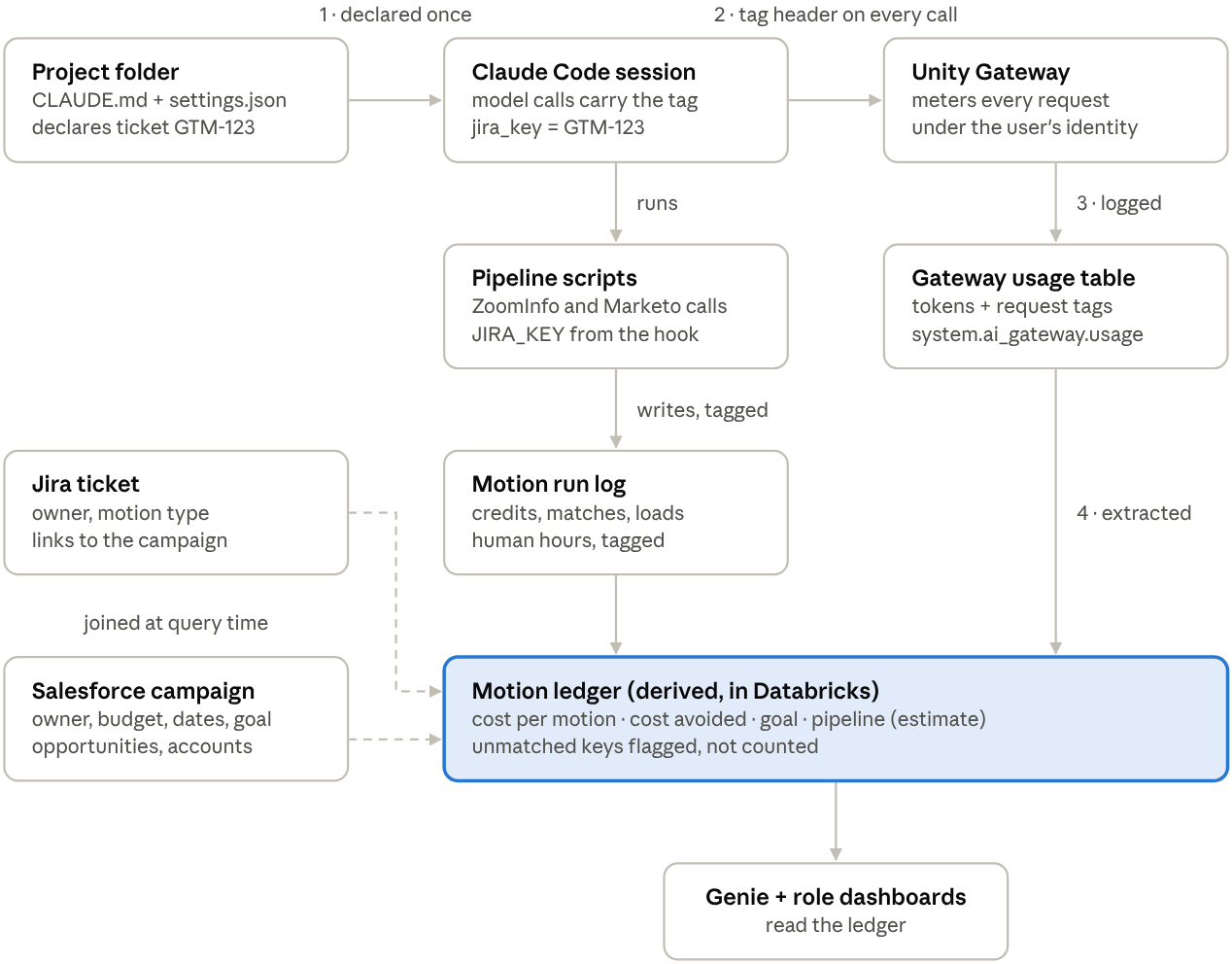
Motion type	Business object	Example run	Defined in
`list-enrich`	Salesforce campaign (event or trade show)	Trade-show list to running campaign	Section 7.1
`account-brief`	Account, through its open opportunities	Daily AE brief; on-call briefing	Sections 4.7, 6
`intent-digest`	Account and the campaigns that fired its signals	Ranked digest to the owning rep	Section 7.2
`icp-score`	Every account and lead; outcome on opportunities	Scoring run plus backtest	Section 7.3
`nba`	Campaign member whose status flipped to Responded	Next-best-action on an attributable touch	Section 4.6
`data-quality`	Campaigns and records flagged by the audit	Daily MOps anomaly digest	Section 6
`report-query`	The campaign or segment asked about	"How many MQLs last week?"	Section 5.5

Work that serves no business object directly (server evals, the headless jobs' scaffolding, dashboard hosting) is booked to a small set of reserved **platform tickets**, so infrastructure is visible as overhead instead of being misread as untethered spend. This is a design choice of this plan, not of the source framework.

3.2 The four stops of an ID

1. **Declared once** in the local project folder: the project `CLAUDE.md` names the motion and the ticket; `.claude/settings.json` holds the tag.
2. **Attached to every outbound model call** as request metadata, through `ANTHROPIC_CUSTOM_HEADERS`. Pipeline scripts read the same key (exported by the SessionStart hook) and write it into the motion run log.
3. **Logged by the gateway** in its usage system table, with tokens and the request tags.
4. **Extracted into the ledger** and joined to the Jira ticket and the Salesforce campaign at query time.

The ticket key rides around the model, from one project file to the ledger



motion ID path · 4 stops, 2 joins at query time

The model never reads or writes the authoritative key: it travels as a header and an environment variable, so prompt text cannot mangle it and context truncation cannot drop it. Jira and Salesforce stay the sources of truth and are joined only at query time.

3.3 Verified mechanics (as validated on Oct 12, 2026)

Piece	What the documentation confirms	Consequence
Per-request tags	Unity AI Gateway accepts a `Databricks-Ai-Gateway-Request-Tags` HTTP header holding a JSON map of string keys and values, and records it in `system.ai_gateway.usage`.	The ticket key can ride as a header.

Piece	What the documentation confirms	Consequence
Older gateway variant	AI Gateway on model serving endpoints records a `usage_context` map from the request body in `system.serving.endpoint_usage` (10 KiB limit).	If we run this variant, the tag goes in the body instead. Confirm first.
Who can query usage	Both usage tables are admin-only by default.	We need a grant or a filtered view.
Tagging from Claude Code	`ANTHROPIC_CUSTOM_HEADERS` adds custom headers to every model request; it can be set in a project's `.claude/settings.json` env block and is reapplied mid-session when the file changes.	A per-project settings file stamps every call with no prompt text involved.
Coding-agent metadata	Since September 2026 the gateway usage table also records coding-agent session metadata and tokens written to the 5-minute and 1-hour caches; unified tracing shows the tools and skills used alongside each request.	Cost per skill and per cache behavior is readable without extra instrumentation.

A project settings file then looks like this (the exact header depends on the gateway variant):

```
{
  "env": {
    "ANTHROPIC_BASE_URL": "https://<workspace>/ai-gateway/anthropic",
    "ENABLE_PROMPT_CACHING_1H": "1",
    "ANTHROPIC_CUSTOM_HEADERS": "Databricks-Ai-Gateway-Request-Tags:
{\"jira_key\":\"GTM-123\",\"motion_type\":\"list-enrich\"}"
  }
}
```

3.4 Data model

Every call stamps exactly one thing, the Jira key; everything else is reached by walking the chain at query time.

Entity	Key	Joins to	Holds
Jira ticket	`jira_key`	Tags gateway usage and run logs; links to one Salesforce campaign or opportunity	Owner, `motion_type`, `sf_campaign_id`, `sf_opportunity_id`
Gateway usage	request row	`request_tags['jira_key']`	Request time, requester, request tags, input and output tokens
Motion run log	`run_id`	`jira_key`	Records in, matched internally, enriched externally, ZoomInfo credits, records loaded, human hours
Salesforce campaign	`campaign_id`	Marketo program (1:1 sync); opportunities it influences	Owner, budget, start date, end date, goal
Marketo program	`program_id`	`sf_campaign_id`	Program membership and statuses
Salesforce opportunity	`opportunity_id`	`account_id`	Amount, stage
Salesforce account	`account_id`		Reached through opportunities, never stamped directly

Design decisions behind the model:

- **Ticket as the join key, not a Salesforce ID.** It decouples work from systems: a call never needs to know whether it serves a campaign, an opportunity or a Marketo program. One enforceable rule (all AI work ties to a ticket) instead of five.
- **Pointer local, truth remote.** The project file holds only the pointer (this run is ticket X). Ticket-to-campaign relationships stay in Jira and Salesforce and are resolved by joins, so nothing goes stale on a laptop.
- **Campaigns and opportunities carry the dollars.** Accounts are reached through opportunities.
- **Flat tag shape:** object type, object ID, optional human-readable name. Event tagging, not a config tree.

- **Validation:** every key in the ledger is checked against a real Jira ticket and Salesforce record. Anything unmatched is flagged, not counted.

3.5 The ledger query, in sketch

```
-- Cost of AI per motion: tokens grouped by the ticket tag
SELECT
  request_tags['jira_key']           AS jira_key,
  SUM(input_tokens + output_tokens)  AS total_tokens,
  COUNT(*)                           AS requests
FROM system.ai_gateway.usage
WHERE request_tags['jira_key'] IS NOT NULL
GROUP BY 1;
-- Then join jira_key -> Jira ticket -> Salesforce campaign,
-- and add ZoomInfo credits and human hours from the motion run log.
```

Column names are illustrative; confirm them against the real table schema before building. The full ledger contract is in section 8.

3.6 ROI model

The ROI of AI is not the token cost. It is the cost and time AI removes from the motion. Tokens alone are cheap and make a weak story, so we measure the whole motion.

Cost side: total motion cost.

```
\text{Motion cost} = \text{AI tokens} \times \text{gateway rate} +
\text{ZoomInfo credits} \times \text{credit rate} + \text{human hours} \times
\text{loaded hourly rate}
```

The AI-token term can also be estimated before a run, from the preload size and the planned calls (section 4.5). Databricks compute (SQL warehouse time, Genie DBUs billed to service principals) belongs in the motion cost too; how to attribute it to a ticket is an open question in section 10.

Cost avoided is reported beside it: ZoomInfo credits not spent because records matched internally (records matched internally × credits per enrichment × credit rate).

Return side: three honest numbers.

1. **Total motion cost**, from the ledger.
2. **Goal attainment** against the campaign's own stated goal, which is defensible because it was set before launch.

3. **Influenced pipeline**: opportunities tied to the campaign, explicitly labeled an estimate. Saying so builds credibility.

The credit rule for influenced pipeline (whether marketing and sales each get credit when both touch an opportunity, influenced versus sourced) must be agreed before any revenue number is shown. The ledger can support either.

3.7 The comparison that makes the case

Same list, two ways. The proof fills this table with real figures; until then the cells stay empty rather than guessed.

Measure	Manual way (today)	Governed way (proof)
Records in		
Matched internally (no purchase)		
ZoomInfo credits spent		
Human hours, list to live campaign		
AI tokens (now visible per campaign)		
Days from show to first touch		
Tied to a Salesforce campaign with goal		
Influenced pipeline (estimate)		

3.8 Reading the ledger across motions

Pattern	What it means	Action
High AI, low hours, goal met	The model to replicate	Fund more of it
Low AI, high hours	Done by hand, or in a tool we can't see	Next target for AI
High AI, no business object	Untethered spend	Question it
High AI, goal missed	Spend without return	Fix or stop the motion

Low AI usage on a motion is a question, not a virtue: it is either manual work, usually the most expensive way, or work happening outside the gateway. Either way it marks where to look next.

3.9 The measurement boundary

Anything that routes around the gateway is invisible to this telemetry, and we say so. The Atlassian MCP already runs outside it, which is fine because Jira is a dimension we join on, not a source we meter. Mapping what else bypasses the gateway defines the true edge of what we can measure, and is part of discovery.

4. GTM context architecture: spend less by knowing more

The context architecture lowers the cost of every motion by moving knowledge upstream into preloaded local rule files, so live lookups become the exception. It also makes the campaign node the shared anchor every person's AI reasons from, which is what lets the ledger trust that two motions on one campaign mean the same thing.

4.1 Thesis and role in the motion model

Most of the value in a sales or marketing interaction comes from context the model should already carry, not from querying live systems mid-task. The design is a local, rules-based `CLAUDE.md` stack: tiered markdown files that tell the model who it serves, what matters and how to behave, assembled on each person's machine before anything reaches the gateway. "Wiki" and "knowledge graph" describe what those files contain and how they link; the engine is a mechanism Claude Code already provides.

Three pillars carry it, and each maps to a ledger effect:

Pillar	What it means	Effect on cost per motion
Structural alignment	Every person's AI anchors on the same shared nodes, above all the campaign node, so sales and marketing reason from one context.	Motions on one campaign are comparable; less rework from misaligned output.
Governed autonomy	Every node carries its own rules: refresh cadence, who may change it, whether changes cascade, and what it costs to resolve. People adopt updates; nothing is imposed silently.	Stale or conflicting context is caught before it spends.
Predictable cost	Context is preloaded and tool calls are pre-declared, so an answer's cost is estimated before it runs.	Each motion has a forecast to compare against the ledger's actual.

The working bet: preloaded context lets the first prompt one-shot a usable output more accurately, faster and cheaper than retrieval by tool call. The ledger is how we test that bet.

4.2 The CLAUDE.md tier stack

Every `CLAUDE.md` from the filesystem root down to the working folder loads at launch and is concatenated, broader first, so a soul file placed above all project folders is inherited by every project without any import.

Tier	File and location	When it loads	What it holds
Company policy (optional)	Managed `CLAUDE.md` deployed by IT (macOS: `/Library/Application Support/ClaudeCode/CLAUDE.md`)	Every session on the machine; cannot be excluded	Compliance and data-handling rules

Tier	File and location	When it loads	What it holds
1 Soul	`~/gtm/CLAUDE.md`, above all project spaces (or `~/.claude/CLAUDE.md` for every project)	At launch, for any project under it	Mission, goals, values, primary sources, people in daily orbit, routing order, and the standing motion rules: check internal first, every motion needs a ticket, IDs never go in prompts
2 Project	`~/gtm/<project>/CLAUDE.md`	At launch in that project	The motion this folder serves and its ticket; project goals, conventions, state; which wiki slices and pre-declared calls apply
2b Personal per project	`CLAUDE.local.md` beside it, gitignored	At launch, after the project file	Private notes for that project
3–4 Scoped knowledge	`.claude/rules/*.md` with `paths:` frontmatter, or a subfolder's own `CLAUDE.md`	Only when Claude reads or edits a matching file	Account, campaign or segment slices of the graph

Five documented behaviors shape the build:

- **Size.** Target under 200 lines per `CLAUDE.md`; longer files cost context and lower adherence. `@path` imports organize a file but do not reduce cost, because imports also load at launch (up to four hops deep).
- **Scoped loading is native.** Path-scoped rules and subfolder `CLAUDE.md` files load on demand. Put each account's node under `accounts/<account>/` and its rules load only when that account is touched.

- **Guidance, not enforcement.** `CLAUDE.md` arrives as a user message after the system prompt. Anything that must happen belongs in a hook: a SessionStart hook can inject a computed subgraph as `additionalContext` (capped at 10,000 characters per field; overflow spills to a file Claude is not told to read), and a PreToolUse hook can deny a call outside the plan.
- **AGENTS.md.** Claude reads `AGENTS.md` only when no `CLAUDE.md` exists on the path. For colleagues on other agents, keep one `AGENTS.md` and import it with `@AGENTS.md`.
- **Plugins cannot ship CLAUDE.md files.** A plugin bundles skills, agents, hooks and MCP servers, so the bootstrap skill must write the soul and project files itself and merge into existing ones without clobbering them.

```
~/gtm/
├── CLAUDE.md                      # Tier 1 soul (loads for every project
below)
├── wiki/                          # Tier 3 global base (shipped) + Tier 4
local overlay
|   ├── market/campaigns/          # campaign nodes (keystone)
|   ├── market/accounts/<acct>/    # persistent account nodes
|   └── local/                     # private; promoted only through review
├── trade-show-q4/                 # one folder per motion
|   ├── CLAUDE.md                  # Tier 2: motion + ticket GTM-123
|   └── .claude/settings.json      # tag header for every model call
├── account-briefs/
|   ├── CLAUDE.md
|   └── .claude/rules/acct.md      # paths: frontmatter, loads only on
matching files
└── campaign-planning/
    └── CLAUDE.md
```

One layout choice is open. Path-scoped rules trigger on files inside the working folder. A wiki beside the projects is reachable with `--add-dir`, but its `CLAUDE.md` files load only when `CLAUDE_CODE_ADDITIONAL_DIRECTORIES_CLAUDE_MD=1` is set. The bootstrap skill either sets that variable or places the wiki inside each project folder.

4.3 Knowledge layers and governance

Position in the stack is the governance: slow, authoritative layers sit high and load whole; fast, personal layers sit low and load as scoped slices.

Layer	Contents	Refresh	Who may change it	Cascades?	How it preloads
World	Regulation, macro, category shifts	Slow, on review	Curators of the global base	No; adopted at quarterly refactor	Small, loads whole
Org	Structure, roles, teams, Jira mapped to owners	Quarterly, or on reorg	Global base owners	Adopted at refactor	Whole if small, else by team
Market: campaigns	Campaign nodes from the crosswalk: IDs, intent, attributable event, NBA	Daily, machine-validated against Salesforce	System of record only	Global updates daily; users adopt	The campaigns a task touches
Market: accounts	Who is involved, touches, open opportunities, stage	Event-driven, at least daily, possibly hourly	System of record plus validated enrichment	Updates the node, not users' notes	One account subgraph per task
Local overlay	A person's own Salesforce, Jira, Slack, Confluence, Drive context	Continuous	That person	Never leaves the machine unless promoted	Scoped by project
Calendar clock	Fiscal period, quarter-end, cycle stage	Calendar events	Finance and leadership	Re-colors interpretation company-wide	Always, a few lines

Three rules keep the layers honest:

- **Two clocks.** Events (an attributable touch, a stage change) refresh facts. The fiscal calendar changes how they should be read, such as quarter-end urgency, even when nothing on the account moved.
- **Validated versus conjecture.** Nodes and edges carry a validation flag. Sourced connections are facts; predictive connections are shown as worth investigating, never as truth.
- **Refactor records lineage.** Each local node records which global node and version it attaches to, so the quarterly refactor reconciles instead of overwriting.

Executive anchor. Executive direction is not a layer inside the wiki; it is an external anchor the wiki points to. Strategic intent (fiscal-year goals, priorities) links down to tangible execution (Jira epics and tickets). A campaign, account play or epic node carries a pointer such as `serves: FY27-priority-2`. Because every motion is already a Jira ticket under

an epic, the ledger can roll cost and return up to the executive priority each motion serves. The soul file loads only the anchor's one-line summary; detail is fetched when a task names a priority.

4.4 Retrieval routing

Route by the nature of the knowledge: the wiki holds the why and what-matters, Databricks holds rollups, Salesforce holds the precise current fact.

Kind of knowledge	Example	Lives in	Reaches the model by	Fresh as
Stable interpretive	Mission, ICP definition, what good looks like	Soul and global base	Preloaded at launch	Quarterly refactor
Moving interpretive	This quarter's pipeline target, a campaign's conversion threshold, this month's definition of qualified	Market layer	Preloaded, refreshed on its own cadence	Daily
Aggregates	Pipeline rollups, campaign performance, trends	Databricks CDP tables	A pre-computed snapshot node; live Genie only for a new cut	Last pipeline run
Single live record	Exact stage or amount on one opportunity now	Salesforce	Live call via Salesforce Hosted MCP, last resort	Real time
Personal	My Slack thread, my notes on an account	Local overlay	Scoped preload by project	Continuous

The wiki never stores live Salesforce rows. It stores query intent and interpretation: which object and fields to fetch, which path to send it down, and how to read what comes back. **Lookup order**, written into the soul file: preloaded context, then local wiki, then the cache (section 5.5), then Databricks (a Unity Catalog function or Genie Agent), then live Salesforce through the gateway.

4.5 The wiki as planner, and the per-task cost model

The preloaded markdown carries a plan of investigation as well as knowledge: for each task, the exact tool calls that may be needed, each mapped to a skill, each with a condition and a budget. A bounded plan turns cost from a random variable into a sum you can estimate before running.

```
task: account-briefing          # = motion_type account-brief
preload: [soul, campaigns/<active>, accounts/<acct>, calendar]
calls:
  - id: opp-status
    skill: sf-live-record       # Salesforce Hosted MCP, sobject-reads
    when: account node older than 24h OR opportunity missing
    budget: 1 call
  - id: touch-rollup
    skill: uc-campaign-touches  # Unity Catalog function over CDP tables
    when: no touch snapshot in last 7 days
    budget: 1 call
escape_hatch: ask before any unplanned call
```

Claude Code sends a new model request for every step of its tool loop, re-sending the whole context each time. The cost of one task is therefore:

```
\text{Cost} \approx w \cdot P + (S-1)\, r \cdot P + \sum_{s=1}^{S}\left( i
\cdot N_s + o \cdot O_s \right) + \sum_{c=1}^{K} \text{tool}_c
```

P is the preloaded prefix in tokens, S the number of model steps (one plus the number of tool rounds), N and O the new input and output tokens per step, K the planned calls, and w, r, i, o the per-token prices for cache write, cache read, input and output. On the Claude API a 5-minute cache write costs 1.25× base input, a 1-hour write 2×, and a cache read 0.1× (0.05× on Opus 5.5). Tool costs are SQL warehouse time, Genie DBUs where billed, or Salesforce API requests.

What this means in practice:

- **Every tool round re-sends the preload,** so caching matters more than prefix size. A large, stable prefix is cheap after step one; a prefix that changes mid-session is expensive.
- **Keep the prefix byte-stable within a session.** Refresh nodes between sessions, not during them.

- **Resumed sessions pay again.** The default cache expires after 5 minutes. Databricks' documented settings turn on the 1-hour cache (`ENABLE_PROMPT_CACHING_1H=1`). On resume, the SessionStart hook reports `prompt_cache_likely_expired` and `estimated_cache_write_usd`, which a hook can surface before the first request.
- **Conditional calls keep most tasks at zero calls.** A fresh preload satisfies the task with S = 1.
- **Estimate before running; enforce the plan.** A hook sums the plan's budgets and shows the expected cost; a PreToolUse hook returns `ask` for an unplanned call, so the model flags a gap instead of silently spending.
- **Measure after, per motion.** Tokens land in `system.ai_gateway.usage` with the ticket tag; Genie usage lands in `system.billing.usage` with `billing_origin_product = 'GENIE'`. Comparing each motion's forecast with its ledger actual is how the wiki earns its keep.

4.6 The campaign keystone and next-best-action

One governed campaign row holds identity and intent: the IDs that tie Salesforce to Marketo, the action the campaign wants the target to take, the event that proves they took it, and what to do next. In the motion model it is also the join path from a Jira ticket to revenue.

Field	Example	Notes
Canonical campaign ID	FY27-WEB-014	System-independent key; never join on campaign name
Salesforce Campaign ID	701…	Join to CampaignMember and opportunity attribution
Marketo Program ID	4521	Marketo syncs one program to exactly one campaign
Lead source	Web Form	From the governed lead-source list
Desired action	Download the guide through the gated form	What the campaign asks the target to do
Attributable event	Member status Sent → Responded, triggered by a form fill	Fires next-best-action

Field	Example	Notes
Next-best-action	Route to SDR within 24 hours with talk track B	Read directly by the engine
Serves priority	FY27-priority-2	Pointer to the executive anchor
Owner, budget, dates, goal	Set before the first touch	The return side of the ledger

Two supporting tables:

- **Identity crosswalk.** Many duplicates share an email, and email alone is not enough. One universal person key gathers every lead ID and contact ID ever attached to a person, plus other identifiers.
- **Lead source dimension.** One centrally governed canonical list, to avoid spelling drift. It feeds attribution and the ICP loop.

Where the crosswalk lives. A metadata table in Salesforce, mirrored to Databricks. Custom Metadata Types are deployable configuration whose Apex queries do not count toward SOQL governor limits, but records change by deployment; a custom object changes by ordinary record edits, which suits campaigns created weekly. Pick by how often marketing ops creates campaigns.

What the systems guarantee (validated): Marketo maps one program to exactly one Salesforce campaign; program statuses must match campaign member statuses before sync, and member status changes sync both ways. Channel and period cost sync one way to Campaign Type and Actual Cost. If a program member does not exist in Salesforce, Marketo creates the lead there. Salesforce sets `HasResponded = true` and `FirstRespondedDate` when a member's status is one flagged as Responded: the clean, native definition of an attributable event. Program-to-campaign sync runs through a queue on a cycle of roughly seven minutes (per community reports), so "real time" means minutes.

Ingestion: Marketo is the single front door. All forms are Marketo forms; data enters Marketo first, then flows to Salesforce through the native sync, not dual submission. Marketo de-duplicates on email for form fills but can still create duplicates through API inserts, CRM inserts or email-less forms, so the identity crosswalk remains necessary.

Next-best-action engine: last-touch triggered, full-history informed.

1. **Trigger.** A campaign member's status changes to a value flagged as Responded. Campaign membership is the trigger vocabulary.
2. **Look up intent.** Read the campaign's crosswalk row: desired action, attributable-event definition, configured NBA.
3. **Assemble context.** The account node, its validated connections, the campaign node, the calendar clock and the executive pointer, as one scoped subgraph.
4. **Recommend.** Apply the configured action to this account's arc.
5. **Measure.** Score the touch and outcome against the ICP; sales qualification feeds back whether the ICP and high-intent signals are right.

Each NBA computation is itself an `nba` motion run, tagged to the campaign that fired it, so its cost lands on the campaign that earns its return.

4.7 Genie and Salesforce paths

Genie can be invoked as a tool, but the cost offload only works when a person is the caller.

Path	Endpoint	Behavior	Best for
Genie Agent managed MCP server	`https://<workspace>/api/2.0/mcp/genie/{genie_space_id}`, OAuth scope `genie`	Read-only; one question in, SQL run, result back; no conversation history	Claude Code calling one curated agent of up to 50 tables
Genie One (MCP Service in Unity Gateway)	`https://<workspace>/ai-gateway/mcp-services/system.ai.genie_one_mcp`, scope `ai-gateway`	Searches across Genie Agents and Unity Catalog; grounded answer	Broad questions when the right agent is unknown
Genie Conversation API	`POST /api/2.0/genie/spaces/{space_id}/start-conversation`, then poll and fetch	Asynchronous, stateful; poll with backoff up to one minute	Scheduled jobs and custom code

Read MCP tool names at runtime with `tools/list` rather than hard-coding them.

Cost	What it covers	Free?
Genie LLM usage	The model that turns a question into SQL	150 DBUs per user per month (about $10.50 in US East); Genie One and Genie Agents free for users through January 31, 2027; service principals billed from the first DBU
SQL warehouse compute	Running the generated query	Never free
Unity Catalog functions	Predefined SQL run as a tool	Serverless general compute

Genie spend is tracked under `billing_origin_product = 'GENIE'` in `system.billing.usage`, with the surface in `usage_metadata.genie.surface` and the caller in `identity_metadata.run_as`. Genie budgets on the Unity Gateway track LLM usage only, not warehouse compute.

Role in the plan: for fixed queries, skip Genie and run plain SQL or a Unity Catalog function, which has no LLM cost and a known cost per call. Pre-declared plan calls prefer functions. Live Genie calls from Claude Code run as the person and are a reasonable on-demand path for new cuts.

	Live: Salesforce Hosted MCP	Copy: Databricks CDP
How	`sobject-reads` server at `https://api.salesforce.com/platform/mcp/v1/platform/sobject-reads`: discovery, query, search, relationship traversal, no writes	Lakeflow Connect lands objects as Delta tables
Freshness	Real time	Pipeline schedule; formula fields need a full snapshot by default
Permissions	Per-user OAuth 2.0 with PKCE; field-level security and sharing rules	Unity Catalog grants, designed to match Salesforce visibility

	Live: Salesforce Hosted MCP	Copy: Databricks CDP
Cost and limits	Plan as if it counts toward the daily API allocation (to confirm): 100,000 plus 1,000 per license per rolling 24 hours on Enterprise Edition	Warehouse compute; no Salesforce API load
Use for	"What is the stage on this opportunity right now?"	Rollups, campaign performance, trends

Connector caveats: hard deletes on soft-delete tables are not captured, and non-formula fields can update earlier than formula fields within the same cadence, so rollups on formula fields note their as-of time.

4.8 Identity and permissions

The permission model is mostly inherited: live calls run as the caller, and local data never leaves the machine.

Path	Enforcement
Local overlay	Physical: the data sits on the person's machine
Databricks managed MCP (Genie, UC functions, SQL)	On-behalf-of-user OAuth; Unity Catalog permissions on every request
Unity Gateway MCP Services	Caller holds `EXECUTE` on the service plus `USE CATALOG` and `USE SCHEMA`; per-user OAuth U2M available; every call logged. A service configured with a shared principal does not inherit access.
Salesforce Hosted MCP	Per-user OAuth with PKCE; field-level security and sharing rules
Shared global layer	Nothing automatic: the same nodes ship to everyone

The shared-layer rule. A node enters the global base only if its content is safe for every recipient. A summary built from a restricted opportunity would leak it to people who cannot query the source. Promotion from local to global therefore has two tests: is it validated, and is it permission-free.

Operational notes: Databricks recommends OAuth for team use, and the `ug` CLI configures Claude Code's models, MCP servers and skills in one step. Databricks MCP endpoints do not support dynamic client registration, so an admin creates the OAuth app once. Workspace IP access lists must allow client addresses. Per-user consent happens once per MCP Service.

5. The governed harness and the gateway

The harness makes the motion stamp unforgettable and the governed path the easy path; the gateway makes the stamp measurable. Two layers do the work: a local harness on each laptop, which knows the task and decides what is allowed and what is already answered, and the company's Databricks Unity Gateway, which holds OAuth identity, the hard cap, models, MCP tools, Genie and the shared cache. Only two things cross the line between them: cache misses and calls the hooks allow.

5.1 The gateway as single checkpoint

Every model completion and every MCP tool call routes through one gateway, so its caps cover total spend and its log is complete by construction. Claude Code is pointed at it with `ug claude`, or by setting `ANTHROPIC_BASE_URL` to `https://<workspace>/ai-gateway/anthropic` together with a Databricks token, the model's Unity Catalog name and `ENABLE_PROMPT_CACHING_1H=1`. MCP services are added with `ug mcp add`, or `claude mcp add --transport http` against `/ai-gateway/mcp-services/<catalog>.<schema>.<service>`. The base URL is pinned in managed settings so users cannot reroute.

In the motion model the gateway's hard cap is the org ceiling: a loose backstop sized to total spend, not the instrument that allocates it. Allocation happens per motion, from the ledger.

5.2 Three tiers of governance, re-keyed to the motion

Tier	Owner	Where it lives	Mechanism	What it controls	Bypassable by user?
Org	Platform and AI gateway owners	Unity Gateway; Claude Code managed settings	Hard ceiling, rate limits, OAuth identity, MCP routing	Total spend; which models and MCP servers exist at all	No

Tier	Owner	Where it lives	Mechanism	What it controls	Bypassable by user?
Team	GTM ops and team leads	A versioned Claude Code plugin per role	Hooks, skills, subagent definitions, permission rules	Which tools and models a role uses; the ticket requirement; per-motion budgets	No, if delivered as managed config
User	Each person	User `CLAUDE.md`, user settings, personal hooks	Context, routing instructions, self-imposed limits	Waste reduction, local-first retrieval	Yes, by design

The team tier changes most under the motion thesis. Its budget gate reads the **motion's** budget (from the ticket's campaign) instead of a per-person daily allowance. Role still decides which tools are enabled, because that is an access question, not a cost one.

Precedence: most restrictive wins.

1. The org ceiling is absolute. A budget, model or MCP server the gateway disallows is unavailable regardless of any harness config.
2. Deny beats allow at every tier, matching how Claude Code already evaluates permission rules.
3. Budgets take the minimum: effective budget = min(org cap, motion budget, user cap).
4. Grants are explicit and bounded. When a team must grant a capability a user turned off, such as a required enrichment tool, it uses a small, reviewed allowlist delivered as managed config, valid only inside the org ceiling.
5. Conflicts are logged, not silent. The deny reason names which tier decided and why.

One honest limit: harness hooks see tool calls, not token counts. Precise model spend is metered at the gateway; harness budgets are approximations (call counts, estimated cost per skill, usage read from the session transcript) that catch waste early and give a readable reason. Tool weights are recalibrated monthly against gateway actuals.

5.3 Harness components

Every component has one governance job, and the whole set ships as one versioned plugin.

Component	Lives in	Job	Cost lever
LLM wiki	`~/gtm/wiki` (section 4)	Definitions, join logic, query intent	Replaces discovery tool calls
CLAUDE.md tiers	Soul and project files	Intent, motion and ticket, routing order, model tier	Right-sizes the model per task
Project settings	`.claude/settings.json`	The ticket tag on every model call	Makes every token attributable
`policy.yaml`	`~/.claude/harness/`	Machine-readable budgets and tiers	Source of truth the hooks read
Hooks	Plugin `hooks/`	Inject, gate, block, log	Makes declarations binding
Skills	Plugin `skills/`	Packaged motions with known call counts	Known calls and tokens per run
Subagents	Plugin `agents/`	Retrieval, matching, formatting, lint	Cheap tier, isolated context
Commands	Plugin `commands/`	Curated front doors, e.g. `/enrich-list GTM-123`	No free-form costly prompts
Local cache	`~/.claude/harness/cache.db`	Hot recent answers	Zero gateway cost on a hit
Central cache	Delta table plus vector index	Shared results with freshness	One query serves the team

Declarations: prose for the model, YAML for the hooks. Skill frontmatter pins the model and tools; custom keys are read by hooks.

```
# skills/report-query/SKILL.md
---
name: report-query
description: Answer Salesforce reporting questions via cache, then a UC
function, then Genie
model: haiku
allowed-tools: mcp__genie__*, Read
x-motion-type: report-query   # custom keys, read by hooks
x-effort: low
x-max-calls: 3
x-budget-tokens: 40000
---
```

```
# ~/.claude/harness/policy.yaml
version: 1
routing: [preload, local, cache, gateway]
require_ticket:
  write_tools: block      # ZoomInfo, Marketo, Salesforce writes
  model_calls: warn       # untagged sessions are flagged, not stopped
motion_budget:
  source: campaign        # AI budget read from the ticket's campaign
  warn_at_pct: [__]
  block_at_pct: [__]
tiers:
  report-query: {max: haiku}
  account-brief: {max: sonnet}
  default: {max: sonnet}
mode: warn                # warn | block
```

`[__]` marks thresholds set from Phase 2 telemetry, not guessed up front.

A subagent for routine matching on a smaller model, with read-only tools (tool names illustrative):

```
---
name: enrichment-matcher
description: Resolves partial event-list rows against Databricks, then
ZoomInfo. Use for identity matching, not writes.
model: haiku
tools: mcp__databricks__query, mcp__zoominfo__search_contacts
---
Match each row on name + company + title + region.
Return a confidence score 0-1 and the evidence for it.
Never write to Salesforce. Never invent an email or phone number.
```

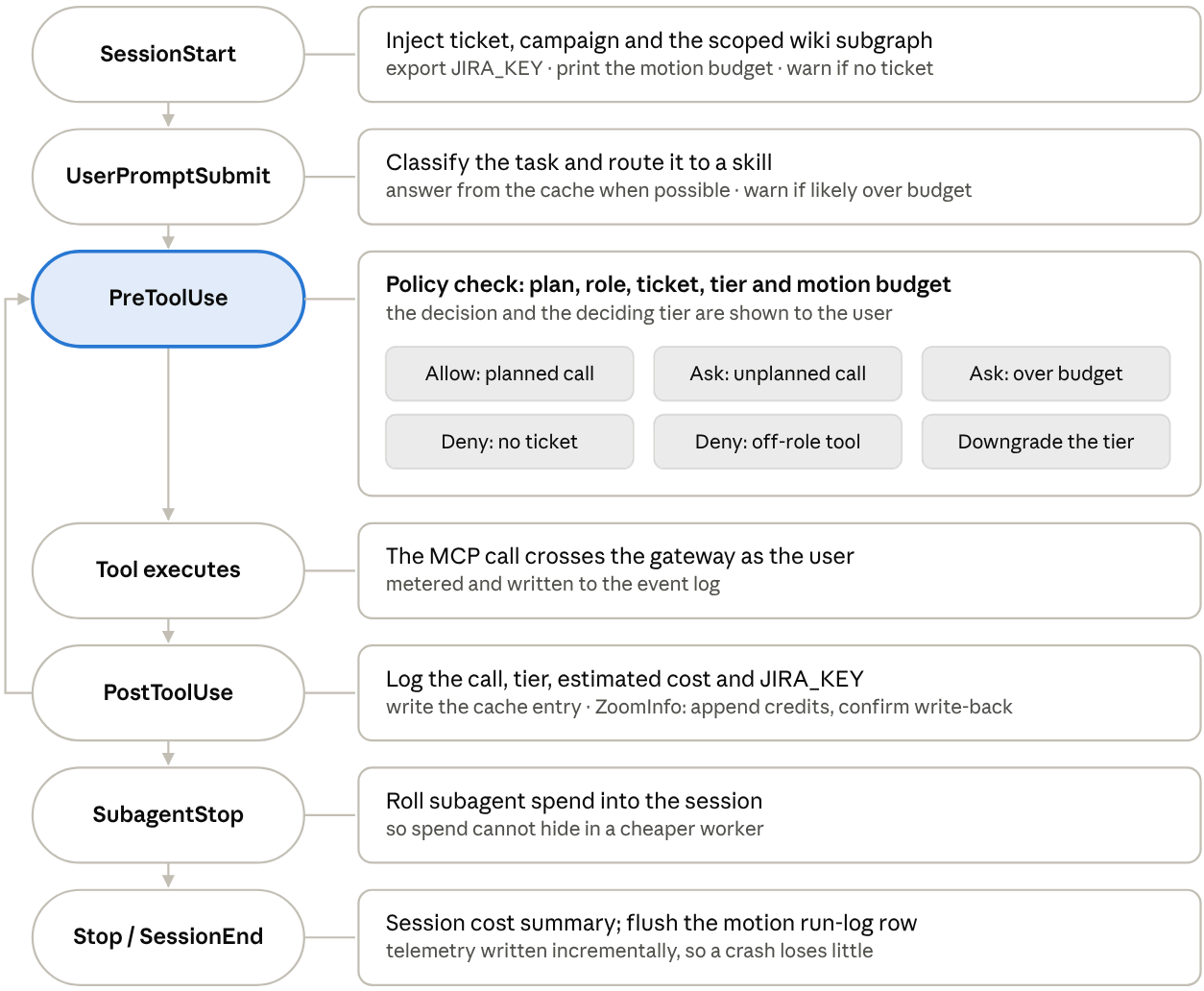
5.4 Hooks: where governance stops being a suggestion

Tagging is a setting, not a hook: hooks act on tool calls (allow, deny, rewrite inputs, add context) and do not add headers to model requests. Hooks do what they are good at, refusing ungoverned work. This is the unified inventory across the source designs.

#	Event	Matcher	What it does	Fails	Ships in
1	`PreToolUse`	ZoomInfo and Marketo write tools	Ticket required: deny when the project has no valid ticket (`onFailure: "block"`)	Closed	Proof hardening
2	`SessionStart`	all	Read the project file; add ticket, campaign and scoped wiki subgraph as context (under 10,000 characters); export `JIRA_KEY` via `CLAUDE_ENV_FILE` ; print the motion's remaining budget; warn if no ticket (this event cannot block)	Open	Phase 1
3	`UserPromptExpansion`	`/enrich-list` and other motion commands	Command gate: block the motion command when no ticket is declared	Closed	Proof hardening
4	`UserPromptSubmit`	all	Classify the task, route to a skill, answer from cache when possible; warn when likely over budget	Open	Cache phase

#	Event	Matcher	What it does	Fails	Ships in
5	`PreToolUse`	ZoomInfo	Check internal first: deny enrichment when the run log shows no internal match step for this run	Closed	Next
6	`PreToolUse`	`mcp__.*`	Plan guard and budget gate: allow planned calls, `ask` on unplanned ones; deny out-of-role tools; check the motion budget; deny or downgrade a model tier above the declared one	Writes closed, reads open	Govern phase, warn first
7	`PostToolUse`	ZoomInfo	Write-back: append credits used to the run log; confirm enriched records were written back to Databricks	Open	Next
8	`PostToolUse`	`mcp__.*`	Log the call, tier and estimated cost with `JIRA_KEY` to the local ledger; write the cache entry	Open	Phase 1
9	`SubagentStop`	all	Roll subagent spend into the session, so spend does not hide in subagents	Open	Phase 1
10	`Stop` / `SessionEnd`	all	Session cost summary; flush the motion run-log row and telemetry	Open	Phase 1

Hooks make every declaration binding before a call leaves the laptop



session lifecycle · 7 events, 6 PreToolUse outcomes

The loop on the left runs once per tool round: every call passes the PreToolUse check again, and every result is logged with its ticket before the next step.

The budget gate, re-keyed from person to motion. The policy and the motion's budget snapshot are written by the SessionStart hook; a single machine keeps a SQLite ledger, and a shared team store (Redis, a Databricks table or a gateway-native budget) replaces it later.

```
#!/usr/bin/env python3
import json, sys, os, sqlite3

event = json.load(sys.stdin)
tool = event.get("tool_name", "")
```

```
jira_key = os.environ.get("JIRA_KEY")          # exported at SessionStart
policy = json.load(open(os.path.expanduser("~/.claude/harness/policy.json")))
motion = json.load(open(os.path.expanduser("~/.claude/harness/motion.json")))
# budget snapshot
rules = policy["roles"][policy["users"].get(os.environ.get("USER"),
"default")]

def decide(kind, reason):
    print(json.dumps({"hookSpecificOutput": {
        "hookEventName": "PreToolUse",
        "permissionDecision": kind,            # "deny" or "ask"
        "permissionDecisionReason": reason}}))
    sys.exit(0)

is_write = any(tool.startswith(p) for p in policy["write_tool_prefixes"])
if is_write and not jira_key:
    decide("deny", "[team policy] Write tools need a Jira ticket in this
project.")
if not any(tool.startswith(p) for p in rules["allowed_tool_prefixes"]):
    decide("deny", f"[team policy: role] {tool} is not enabled for your
role.")

db = sqlite3.connect(os.path.expanduser("~/.claude/harness/ledger.db"))
db.execute("CREATE TABLE IF NOT EXISTS calls (jira_key TEXT, tool TEXT,
est_usd REAL)")
spent_local = db.execute("SELECT COALESCE(SUM(est_usd),0) FROM calls WHERE
jira_key=?",
                         (jira_key,)).fetchone()[0]
spent = motion["ledger_spent_usd"] + spent_local   # ledger actual + calls
since snapshot
budget = min(motion["ai_budget_usd"], policy.get("user_cap_usd",
float("inf")))
if spent + rules["tool_cost_usd"].get(tool, 0) > budget:
    if policy["mode"] == "block":
        decide("deny", f"[motion budget: {jira_key}] ${spent:.2f} of
${budget:.2f} used.")
    decide("ask", f"[motion budget: {jira_key}] Over budget; continue?")
sys.exit(0)   # allow; the PostToolUse hook records the call
```

Rules for running hooks safely:

- **Warn before block.** Every gate starts in warn mode and is promoted to block only once logs show it will not fire on legitimate work. False blocks are how harnesses get disabled.
- **Hooks fire on every matching event,** so a bad hook breaks everyone's workflow at once. They are the most powerful and most dangerous piece, and they ship only after the one motion is proven by hand.
- **Where hooks live:** a plugin ships them in `hooks/hooks.json`; teams can also define them in `~/.claude/settings.json`, a project's `.claude/settings.json`, or organization-wide managed settings, which admins can lock with `allowManagedHooksOnly`. Enforced hooks belong in managed settings; hooks in user settings are advisory.
- **Design for each failure mode:** SessionStart injection bloats if scope is too wide; UserPromptSubmit can misclassify the tier; PreToolUse false blocks push people to disable it; PostToolUse adds latency on every call; spend can hide in subagents unless SubagentStop rolls it up; Stop-time telemetry is lost on a crash unless flushed incrementally.

5.5 The tiered query cache (gated on ledger evidence)

Cost should scale with distinct, fresh questions, not with how often people ask. The cache is built only if the ledger shows a meaningful share of motion spend going to near-duplicate questions, typically repeated `report-query` motions against Salesforce reporting. Its home is ultimately the platform team.

One query through the tiers:

1. The resolver normalizes the question ("How many MQLs last week?" becomes `{metric: MQL, window: 2026-W40, seg: ALL}`) and builds a signature.
2. **Local hit and fresh:** answered with zero gateway cost.
3. **Local miss, central hit, source unchanged:** someone else already paid; lookup cost only; the result is promoted to the local cache.
4. **Central miss or stale:** one fresh run (a UC function, or Genie for a new cut), written back with its dependencies and timestamp so the next identical ask is cheap for everyone.

Every hit is logged with the asker's `jira_key`, so the ledger credits cost avoided to the motion that benefited.

The resolver carries the most risk. A false miss costs one query; a false hit puts a wrong number in someone's report. When unsure, miss. Five controls:

1. **Normalize first:** canonical intent (time window, entity, filters, grouping), so most matching is exact structure.
2. **Bind parameters:** window, segment and filters match literally, never by similarity.
3. **Calibrate the threshold:** label real query pairs and set the semantic cutoff at zero false positives, then back off further.
4. **Test adversarially:** minimal pairs, such as the same metric over a different window, or the same window for a different segment.
5. **Observe and reverse:** log every hit with its signature and score, and give users a one-step "re-run fresh".

Invalidation follows the fastest-moving dependency. An entry that depends only on wiki knowledge is version-tied (editing the doc invalidates it). One that depends on live tables compares its entry time with each Delta table's last commit. A blended entry is invalidated on any change. A safety TTL (24 hours is illustrative) backstops all of them and is set per table refresh cadence.

Central cache schema: `harness.query_cache`, one row per answer.

Column	Type	Purpose
`signature`	STRING (PK)	hash(template_id, params, access_scope)
`template_id`	STRING	Which canonical question this answers
`params`	MAP<STRING,STRING>	Window, segment, filters, matched exactly
`access_scope`	STRING	Permission group allowed to read this result
`result`	STRING (JSON)	Rows plus the generated SQL, for audit
`source_tables`	ARRAY<STRING>	Delta tables the answer depends on
`source_versions`	MAP<STRING,BIGINT>	Delta version of each table at write time
`wiki_versions`	MAP<STRING,STRING>	Wiki docs used, for version-tied invalidation
`created_at`	TIMESTAMP	Entry time; drives the TTL backstop
`hit_count`	BIGINT	Savings evidence and hot-set ranking

Column	Type	Purpose
`origin_jira_key`	STRING	The motion that paid for the fresh run (added for the ledger)

The freshness check compares `source_versions` with each table's latest Delta version. `access_scope` in the key prevents leaks across permissions; restricted data is keyed per user or excluded.

	Local hot cache	Central Delta cache
Runs on	Each laptop, in the harness	Databricks, behind the gateway
Holds	Recent hits for this user	Every result, for everyone
Lookup cost	Free, instant	One cheap keyed read
Freshness	Short TTL, re-checks central	Delta commit metadata versus entry time
Ships in	The plugin	A schema we own, then promote to the platform

5.6 Slack as a governed surface

Slack is another client of the gateway, not a parallel brain. Its message plane (DMs, replies, thread posts) runs on Slack's own API; its model and tool plane always goes through the gateway.

1. A user DMs or mentions the app; Slack delivers the event via Socket Mode or the Events API.
2. The app acknowledges within Slack's roughly 3-second window, before doing any work.
3. The app enqueues the job and calls the gateway with the user's identity, never a shared service account, and with the ticket tag of the motion the request serves.
4. The gateway runs the completion and MCP calls under that user's access and that motion's budget.
5. The app posts the result into the thread asynchronously.

Identity mapping is the critical design point: Slack user ID to email to gateway OAuth identity and team role. A single service account would collapse every request into one budget and one set of permissions. The app stays deliberately dumb: no model keys, no direct MCP connections, no business logic.

6. Role dashboards: the MCP app and the lead card

The role dashboards are the read surface of the motion model: one remote MCP server serves role-specific GTM dashboards as MCP Apps, rendered inline in Claude for Marketing Ops, BDRs, AEs and leaders. Role is a filter over motions, so the dashboards come almost free once the ledger and the audit's data foundations exist, which is why they are built after the proof, not before it.

6.1 Goals, non-goals and personas

The goal is one trusted, role-aware view of GTM data inside the tool reps and marketers already use to think. It is not a BI replacement or a write path into source systems.

Goals: a dashboard per role tuned to its daily decisions, opened by asking Claude; one data foundation (deduplicated records, canonical campaign IDs, one account truth); assembled context such as the account brief and the NBA with its reason beside the numbers; row-level access by the viewer's identity, never by what the model passes in; one role end to end first.

Non-goals for v1: writing back to Salesforce or Marketo (actions stay in those systems and their flows); real-time data (daily for most views, hourly where a role needs it); replacing Marketo Measure for attribution or Revenue Cycle Modeler for lifecycle (the dashboards read their outputs).

Role	Daily question	Primary sources	Motions it consumes
Marketing Ops	Is the funnel healthy, and is attribution trustworthy?	Marketo, Salesforce campaigns, Marketo Measure	`data-quality`, `list-enrich`, ledger by campaign
BDR / SDR	Who do I work today, and what do I say?	Marketo MQLs, ZoomInfo intent and firmographics, NBA	`nba`, `intent-digest`

Role	Daily question	Primary sources	Motions it consumes
AE	Which deals and accounts need me this week?	Salesforce opportunities, account NBA, ZoomInfo	`account-brief`, `nba`
Sales / GTM leadership	Where is pipeline coming from, and is it converting? Which motions return?	All three, aggregated, plus the motion ledger	Ledger rollups by motion and priority
Demand gen / ABM	Which programs and accounts are moving?	Marketo programs, account object, campaign crosswalk	`intent-digest`, `icp-score`

6.2 Architecture and data layer

Four parts: scheduled ingestion into Databricks, a read-only MCP server, headless Claude Code jobs, and Claude as the rendering host. Only the ingestion jobs touch source-system APIs. Dashboards read only Databricks gold tables, never source APIs at view time: Marketo enforces a daily API quota and short-window rate limits, and ZoomInfo bills by credit, so live fan-out per open would be slow, costly and fragile.

Bronze (ingestion).

Source	Method	Objects	Cadence
Salesforce	Existing ETL or Lakeflow connector	Lead, Contact, Account, Opportunity, Campaign, CampaignMember, Task, User, UserRole	Hourly
Marketo	Bulk Extract API	Leads, activities, programs, smart campaign membership, custom objects	Daily, with an intraday activity pull if needed
Marketo Measure	Its Salesforce objects or warehouse export	Touchpoints, attribution touchpoints	Daily

Source	Method	Objects	Cadence
ZoomInfo	Bulk enrich and intent export, scoped to known accounts and contacts	Firmographics, technographics, intent topics, contacts	Weekly enrich, daily intent

Silver (conformed) is where the attribution audit's work lives: identity (deduplicated person and account keys with survivorship rules and a log of where duplicates entered); the campaign crosswalk (canonical campaign ID to Marketo program and Salesforce campaign IDs, never joined on name); lifecycle (stage history reconciled between Marketo and Salesforce, lifecycle-blind arrivals flagged); and the enumerated action vocabulary.

Gold (serving) holds one table per view, already at the grain the UI shows: `gold.mops_funnel_daily`, `gold.mops_attribution_by_campaign`, `gold.mops_data_quality`, `gold.bdr_worklist`, `gold.ae_account_brief`, `gold.ai_summaries`, plus `gold.motion_ledger_by_campaign` for the motion views (section 8). Every gold row carries `owner_user_id`, `territory_id` and `refreshed_at`, which drive row-level security and the freshness stamp; rows tied to a campaign also carry `canonical_campaign_id`.

The server queries a SQL warehouse through the SQL Statement API with parameterized SQL. Unity Catalog row filters on `owner_user_id` and `territory_id` enforce scope a second time below the server.

6.3 MCP server and tools

A TypeScript MCP server built with the official MCP Apps SDK, served over Streamable HTTP. `@modelcontextprotocol/sdk` runs the server and `@modelcontextprotocol/ext-apps/server` registers app tools and UI resources; views use `@modelcontextprotocol/ext-apps` (`App`, `PostMessageTransport`). Each dashboard is a Vite single-file build, because the sandbox loads no external URLs. Queries go through `@databricks/sql` or the SQL Statement REST API.

Tool	UI resource	Returns	Notes
`open_dashboard`	none	The caller's role and which dashboard tool to call	Router; role from identity, not arguments

Tool	UI resource	Returns	Notes
`mops_dashboard`	`ui://gtm/mops`	Funnel, attribution, data quality	Filters: date range, segment, canonical campaign ID
`bdr_dashboard`	`ui://gtm/bdr`	Today's worklist with NBA, reason and context	Caller's owned records
`ae_dashboard`	`ui://gtm/ae`	Account briefs and opportunity watchlist	Caller's territory and accounts
`leader_dashboard`	`ui://gtm/leader`	Pipeline sourcing and conversion; cost and return by motion	Aggregates only, no row detail
`get_lead_card`	`ui://gtm/lead-card`	One person: ICP fit, NBA, open opportunities	Model- and app-visible
`get_account_opps`, `kg_get_node`	none (app-only)	Drill-down rows; a knowledge-graph node and its edges	`visibility: ["app"]`: only the View can request them
`get_account_brief`	none	Text brief for one account	Lets Claude reason in chat
`explain_metric`	none	Definition, source table, as-of time	Keeps answers traceable

Contract for each dashboard tool: return `structuredContent` (what the view renders) plus a short text `content` fallback for hosts without MCP Apps; let the view call server tools back through the host for drill-downs without a new turn; let the view push a message into chat ("Draft an email to these five accounts") so Claude acts with the user in the loop; keep payloads pre-aggregated, capped at a few hundred rows and paged.

```
gtm-dashboards-mcp/
  server/
    index.ts        # transport, auth middleware, tool registration
    tools/          # one file per tool
    identity.ts     # token -> user -> role/territory
    warehouse.ts    # parameterized SQL, no string building
  views/
    mops/ bdr/ ae/ leader/ lead-card/   # one Vite app each -> single HTML
file
    shared/         # charts, tables, theme tokens
  jobs/             # headless Claude Code prompts and runners
  CLAUDE.md         # conventions for Claude Code in this repo
```

6.4 The lead card call path

The lead card shows how a rep's question and clicks travel through an MCP App. Protocol names follow the MCP Apps extension, spec version 2026-01-26 (modelcontextprotocol/ext-apps); they were taken from a secondary write-up that tracks the spec and must be confirmed against the official text before building. The gateway as single checkpoint, the cache key, the `data-*` attributes, stable node IDs and the event log are design choices, not spec requirements.

Phase	What happens	Key messages
0 Connect	The host declares MCP Apps support; the gateway forwards under its own credential and filters tools outside the rep's allow-list; the host prefetches the View HTML.	`initialize` with `extensions.io.modelcontextprotocol/ui`, `tools/list`, `resources/read` `ui://gtm/lead-card`

Phase	What happens	Key messages
1 Seed	"Brief me on Jane Doe at Acme": the model calls `get_lead_card`. The gateway checks authN, authZ and quota, then the cache (key = tool + entity ID, valid while `computed_at` is current). A miss queries gold; a hit touches nothing upstream. Every call is appended to the event log.	`tools/call get_lead_card {person_key}`; `CallToolResult` with `content` for the model and `structuredContent` for the View
2 Render	The host creates a sandboxed iframe (`sandbox="allow-scripts"`); the View handshakes, then receives the arguments and the result, renders, and reports its height.	`ui/initialize`, `ui/notifications/initialized`, `ui/notifications/tool-input`, `ui/notifications/tool-result`, `ui/notifications/size-changed`
3 Click	"Other open opportunities": the View reads `account_id` from the clicked element's `data-account-id` and requests `get_account_opps`. The host refuses any tool whose visibility lacks `"app"`, and any call before `initialized`. The call goes out through the gateway; the log records `origin: "app"`.	View `tools/call` over postMessage JSON-RPC, matched by `id`

Phase	What happens	Key messages
4 Knowledge	Clicking the ICP fit score calls `kg_get_node` with `node_id` from `data-kg-node`; the server resolves the wiki node by its stable ID in frontmatter, and `[[links]]` become edges. The View pushes a summary to the model.	`kg_get_node {node_id: "concept:icp-fit", depth: 1}`, `ui/update-model-context`
5 Teardown	The host asks the View to clean up before the iframe is destroyed.	`ui/resource-teardown` request and acknowledgement

```
{
  "tools": [
    { "name": "get_lead_card",    "_meta": { "ui": { "resourceUri":
"ui://gtm/lead-card" } } },
    { "name": "get_account_opps", "_meta": { "ui": { "visibility": ["app"] }
} },
    { "name": "kg_get_node",      "_meta": { "ui": { "visibility": ["app"] }
} }
  ]
}
```

Three design rules matter most:

- **Keep the View's CSP locked.** With `connectDomains: []` the View has no network path of its own and talks only to the host. Listing a domain such as the Salesforce instance would let the card fetch directly from the browser, skipping the host gate, the gateway, the cache and the log.
- **Every element carries the identity of its data:** `data-person-key`, `data-account-id`, `data-opportunity-id`, `data-canonical-campaign-id`, `data-kg-node`, `data-computed-at`. A click is parameterized by real data from the DOM, not by a model guess.
- **The event log is complete by construction** because every call crosses the gateway: `{tool, origin (model | app), caller, cache_hit, latency_ms, jira_key}`. It feeds usage analytics and grounds a local model by retrieval, never by weight updates.

In v1, `get_lead_card` reads gold tables (`gold.bdr_worklist`, `gold.ae_account_brief`) rather than live SOQL, per the reconciliation in section 2. Card usage is tagged to the `account-brief` or `nba` motion it serves.

6.5 Identity, role resolution and security

The signed-in user's identity decides both the role and the rows; the model never does. A role or owner passed as a tool argument is ignored for access.

1. The user adds the server as a custom connector in Claude and signs in with OAuth against the company IdP (Okta or Entra).
2. The server validates the access token on every request and reads the email or IdP subject.
3. `identity.ts` maps the email to the Salesforce `User`, then `UserRole`, `Profile` and territory, from a cached `gold.user_access` table refreshed hourly.
4. A role map translates Salesforce roles to dashboard roles (MOps, BDR, AE, leader), with an explicit override list for edge cases.
5. Every query is parameterized with the resolved `user_id` and `territory_ids`; Unity Catalog row filters enforce the same scope again.

Credentials: the server uses a Databricks service principal with read-only access to gold; it never holds Salesforce, Marketo or ZoomInfo credentials, which live only in the ingestion jobs' secret scope. A manager's broader view is allowed only after a role-hierarchy check.

Data handling: exclude restricted fields (personal phone numbers, anything under financial-services compliance review) from gold entirely; log every tool call with user, tool, filters and row count but never result data; treat any text in a tool result as data, because CRM free text can carry prompt injection, so views escape it and summaries quote rather than follow it; get security and compliance review before broad rollout, including whether the AI usage policy covers CRM data in Claude.

6.6 Dashboard specs by role

Each dashboard answers one daily question, shows its freshness, and ends in an action taken in Salesforce or Marketo, or asked of Claude.

- **Marketing Ops (build first):** funnel by lifecycle stage with conversion and velocity, filterable by segment and canonical campaign ID; attribution by canonical campaign (touchpoints, sourced and influenced pipeline from Marketo Measure, share of opportunities with no attributable touch); data-quality panel (new duplicates by entry

point, lifecycle-blind arrivals, unpopulated ICP fields); MQL quality (by trigger, follow-up time, outcome against ICP fit); campaign drill-down to member progression and sync mismatches; and the motion ledger for each campaign: AI cost, ZoomInfo credits, cost avoided.

- **BDR / SDR:** today's worklist of owned MQLs and high-intent contacts ranked by NBA priority, each row with the NBA action, reason, ICP fit, intent topics and computed time; an account drawer with account-level NBA and buying-group activity (an account play can override a person NBA); action buttons that prompt Claude, such as "Draft a first-touch email for this contact using the account brief".
- **AE:** account watchlist with account NBA, tier, buying-group engagement and intent surges; opportunity panel with stage age, last activity and marketing touches since the last stage change; the daily account brief from `gold.ai_summaries`, with as-of time and sources.
- **Leadership:** pipeline by source and canonical campaign with conversion to closed-won; MQL-to-opportunity conversion by team and segment; the motion ledger rolled up by motion type and executive priority (cost, goal attainment, influenced pipeline labeled as an estimate). Aggregates only.

Shared UI rules: every view shows `refreshed_at` and its source tables (through `explain_metric`); one theme with light and dark support and phone-width layouts, since many reps open these on mobile; empty and error states say what is missing and why.

6.7 Headless Claude Code jobs

Claude Code runs non-interactively (`claude -p`) on a schedule for work that needs judgment. Outputs land in Databricks; dashboards only read them. Each job runs with its motion's ticket tag in `ANTHROPIC_CUSTOM_HEADERS`, so its spend lands in the ledger like any other motion.

Job	Cadence	Reads	Writes	Motion
Account briefs	Daily, before 7am PT	`gold.ae_account_brief`, ZoomInfo intent, recent activity	`gold.ai_summaries` (role = AE)	`account-brief`
BDR worklist notes	Daily	`gold.bdr_worklist`	`gold.ai_summaries` (role = BDR), one line per top record	`nba`

Job	Cadence	Reads	Writes	Motion
Data-quality digest	Daily	`gold.mops_data_quality`	`gold.ai_summaries` (role = MOps), anomalies with counts	`data-quality`
Server evals	Every pull request and nightly	MCP server in a test environment	Pass or fail report in CI	Platform ticket

A CI runner, or a Databricks job calling a container, executes `claude -p "<prompt file>" --output-format json` with tool permissions limited to the job's needs. Jobs read through a read-only Databricks MCP server or SQL tool and write through a single `write_summary` tool that validates schema, role and owner. Prompt files live in `jobs/`, versioned and reviewed like code. Model calls route through the company AI gateway.

Evals: for each role, a fixture user calls its dashboard tool, and the test asserts a valid result shape, rows scoped to that user only, and a present text fallback. A negative test has a BDR fixture pass another owner's ID as an argument; it must get only its own rows. A render test loads each view's HTML in a headless browser with a fixture payload and checks for console errors.

6.8 Deployment, distribution and operations

Deploy the server as a stateless container behind the company's auth boundary, then add its URL to Claude as an org custom connector.

Option	Pros	Cons
Databricks Apps	Close to the data; service principal and Unity Catalog built in	Check support for public HTTPS ingress and OAuth for an external MCP client
Cloud Run, ECS or the internal platform	Standard ops, easy autoscaling, familiar to platform teams	Needs its own network path to the SQL warehouse

Distribution: a Claude org admin adds the server URL as a custom connector; users enable it per chat and sign in once. Each tool description says what it is for and who it is for, so Claude picks the right tool. A short how-to: "Ask Claude: show my dashboard", plus

two or three example prompts per role.

Development loop: install the MCP Apps plugin (`/plugin install mcp-apps@modelcontextprotocol-ext-apps`); build against fixture data, then a dev SQL warehouse; test rendering in Claude Desktop against the local server (MCP Inspector checks tool shapes but does not fully render apps); CI runs evals on every pull request, and merging to main deploys.

Operations: tool latency and error rate per tool, with an alert when gold tables miss their freshness target; warehouse query cost per dashboard open and model cost of the headless jobs, both booked to the ledger; weekly active users by role and tool calls per user as the pilot's adoption metrics.

7. GTM engineering workflows: the motions themselves

The workflows are where AI earns its return, and each one is built as a ticketed motion from day one. Claude Code adds judgment where rules engines fail (is this the same company under another name, does this title fit our buyer, is this signal real intent); the MCP connectors are the hands, versioned skills hold the GTM logic, and the gateway keeps it safe to run org-wide.

7.1 The proof motion: a trade-show list, rebuilt

One real list, run through a governed path, proves cost containment and ROI on the same campaign. It was chosen because everyone recognizes the pain and it touches every system: Databricks, ZoomInfo, Marketo, Salesforce.

Today: someone returns from a trade show with a ragged spreadsheet (inconsistent columns, personal emails or none). Live-event lists are loaded manually, often bypass Salesforce, and arrive with enough to identify someone (name, company, title, region) but no email or phone. The list goes through ZoomInfo blind, is hand-massaged into Marketo, and attribution is an afterthought. Three months later nobody can say what the show produced.

Governed, merging the motion framework's six steps with the event-list resolve workflow:

Step	Tool	What makes it governed	Run log records
0. Declare	Jira, Salesforce, Marketo	Ticket, campaign (owner, budget, dates, goal) and Marketo program created before the first touch; crosswalk row with the canonical campaign ID; project folder carries the tag	`jira_key` , `canonical_campaign_id`
1. Intake and normalize	Databricks	Raw file to one standard staging table regardless of who built the spreadsheet; every row tagged with the canonical campaign ID and `run_id`	`records_in`
2. Check internal first	Databricks SQL written by Claude Code	Deterministic, parameterized match on email, falling back to company plus name; the `enrichment-matcher` subagent judges fuzzy cases (company aliases, title variants, parent and subsidiary names) and returns a match, a confidence score and the evidence. Genie is for human exploration, not this path.	`matched_internal`

Step	Tool	What makes it governed	Run log records
3. Split into buckets	Databricks	Already owned, partial, genuinely new. The split is the cost-containment number: "matched X% internally, paid for the rest".	Bucket counts
4. Enrich the gap only	ZoomInfo API, batched	Only partial and new records are bought; results are written back to the Databricks account and contact tables, so the next list matches more internally	`enriched_external`, `zoominfo_credits`
5. Confidence gate	Skill logic	Above threshold (start conservative, for example 0.85, tuned from review outcomes): proceed. Below: a holding queue in Databricks, never Salesforce leads with blank emails. Never write a fabricated or guessed email.	Held count

Step	Tool	What makes it governed	Run log records
6. Load to Marketo	Marketo via MCP	One clean push into the program. Marketo first, so lifecycle and scoring stamp before the native sync to Salesforce with campaign membership attached. No logic in Marketo: it is an endpoint for a finished list.	`records_loaded`
7. Attribution wiring	Marketo to Salesforce campaign sync	Program and campaign are a 1:1 pair with matching statuses; owner, dollars, dates and goal were set in step 0, not after	Sync confirmed
8. Sweep	Scheduled job, Slack	Retries the holding queue as ZoomInfo data refreshes; rows unresolved past a set age go to a human review list in Slack with the attributes laid out	Resolved later, sent to review

Human hours on the motion are recorded in the run log too, so the manual-versus-governed comparison in section 3.7 can be filled from data.

How the harness enforces it: `/enrich-list GTM-123` is the front door (blocked without a ticket); the `enrich-list` skill runs the steps the same way for everyone; the ticket-required hook denies ZoomInfo and Marketo writes without a valid ticket; the check-internal-first hook denies enrichment until step 2 has run; the write-back hook confirms credits and write-back after every ZoomInfo call. Compliance becomes the lazy option: one command beats building a spreadsheet.

Scope guard: the proof is proof-of-concept scripts on one machine, not a hardened production system. Present it as the pattern, not the product.

Metrics: resolution rate and time to resolve; false-match rate from human review (tunes the threshold); share matched internally and credits avoided; cost per resolved record; days from show to first touch.

7.2 High-intent signal tracking

A scheduled sweep plus event triggers collect signals from Marketo activity (form fills, high-value page visits, campaign responses) and ZoomInfo intent data. Each account is scored on recency, strength and breadth across its buying group.

- The definition of "high intent" lives in a versioned skill, so it is explicit, reviewable and tunable rather than buried in a scoring UI.
- The trigger vocabulary is the attributable campaign-member status change (for example Sent to Responded), defined per campaign in the crosswalk.
- Output: a ranked digest posted to the owning rep's Slack, and the account's next-best-action field updated.
- Ledger: each sweep is an `intent-digest` motion run; value is read from the digest-to-rep-action rate (Salesforce tasks and activity) and from pipeline on the campaigns whose signals fired.

7.3 ICP scoring model

The ICP is a weighted attribute model, not a static filter, and the backtest is what makes it a model rather than a guess.

1. **Define** firmographic, technographic and persona attributes, each with a weight.
2. **Score** every account and lead; store the score in Databricks and sync it to a field.
3. **Backtest** against closed-won and closed-lost outcomes: which attributes actually predicted revenue, and which ICP-fit accounts never converted?
4. **Close the loop:** rep qualification outcomes on MQLs show whether the high-intent signals and the ICP are right; weights are revised from evidence.

The backtest depends on populated attributes, which ties directly to deduplication: duplicates fragment attributes across copies. ICP work is sequenced after dedup and the universal person key. Value is read as backtest lift: the win rate of high-fit versus low-fit accounts.

7.4 Data foundations each motion depends on

The workflows are only as good as the data layer under them, and that layer is what the attribution audit is building.

Foundation	Where it lives	What depends on it
Canonical campaign ID and campaign crosswalk (Salesforce campaign ID, Marketo program ID, lead source, attributable event, NBA)	Salesforce metadata table, mirrored in Databricks	Event-list tagging, intent triggers, attribution, the ledger's join from ticket to revenue
Universal person key (folding every lead ID for a person)	Databricks	Identity resolution, dedup, ICP backtest
Governed lead source dimension	Salesforce and Databricks	Attribution, the ICP loop
Enumerated action vocabulary	Crosswalk	Intent definitions, next-best-action
Marketo-first single front door	Marketo forms and API intake	Dedup; lifecycle stamping before sync
Account node and ABM custom object	Databricks, synced to Marketo named accounts	Account-level intent, account NBA

System roles. Marketo handles capture, engagement and hand-raise origination. Salesforce is the operational system of record and carries the canonical campaign ID. Databricks is the analytical layer: crosswalks, attribution, ICP scoring, the holding queue and the motion ledger.

Write-back discipline. AI-generated narrative goes into task bodies and digests, not structured fields. Structured fields carry only computed values (scores, NBA codes, timestamps), so the measurement surface stays clean.

7.5 Choosing the next motions

After the proof, the ledger picks what comes next. A motion with low AI and high human hours is the next target; a motion with high AI and no business object is questioned; a motion with high AI and a missed goal is fixed or stopped. Candidates already designed

are the intent digest, the ICP backtest, account briefing and the data-quality digest, each shipped with its ticket, its skill and its run log.

8. Instrumentation and the ledger data contract

The whole plan stands on one contract: every surface that spends attaches the Jira key the same way, every log keeps it, and one daily job turns those logs into the motion ledger. The health of that contract is itself a metric: the share of spend that arrives untagged.

8.1 How each surface attaches the stamp

Surface	How the key is attached	Where it lands
Claude Code, interactive	`ANTHROPIC_CUSTOM_HEADERS` in the project's `.claude/settings.json`	Gateway usage table, `request_tags` (or `usage_context` on the older variant)
Pipeline scripts	`JIRA_KEY` exported by the SessionStart hook through `CLAUDE_ENV_FILE`	Motion run log
Harness hooks	`JIRA_KEY` written by the PostToolUse hook with each tool call	Local ledger, later a shared tool-call log
Headless jobs	The job runner sets the same header for the job's motion ticket	Gateway usage table
Slack app	The app sets the header on each gateway call for the motion the request serves	Gateway usage table
MCP app tool calls	The gateway event log records tool, origin, caller and entity; the motion is derived from tool and entity (`get_lead_card` maps to `account-brief`) and the campaign from `canonical_campaign_id`	Gateway event log
Cache hits	The resolver logs the asker's key and the entry's `origin_jira_key`	Cache hit log; credited as cost avoided

Model tokens spent inside the Claude web or desktop app while a dashboard is open do not cross the Databricks gateway unless policy routes them there. Until that is decided, they sit outside the measurement boundary and are reported as such.

8.2 Ledger tables (sketch)

Table and column names are illustrative; the gateway's real schema is confirmed in discovery.

```
-- Rates, effective-dated, so history re-prices correctly
CREATE TABLE gtm_ledger.rate_card (
  rate_type      STRING,     -- 'model_token', 'zoominfo_credit',
'labor_hour', 'dbx_compute'
  rate_key       STRING,     -- model + token class (input, output,
cache_read, cache_write_5m, cache_write_1h)
  usd_per_unit   DECIMAL(18,8),
  effective_from DATE,
  effective_to   DATE
);

-- One row per pipeline run, written as the motion runs
CREATE TABLE gtm_ledger.motion_run_log (
  run_id             STRING,
  jira_key           STRING,
  motion_type        STRING,
  canonical_campaign_id STRING,
  started_at         TIMESTAMP,
  ended_at           TIMESTAMP,
  records_in         INT,
  matched_internal   INT,
  enriched_external  INT,
  zoominfo_credits   INT,
  records_loaded     INT,
  records_held       INT,
  human_hours        DOUBLE
);

-- One row per governed tool call (harness PostToolUse + gateway event log)
CREATE TABLE gtm_ledger.tool_call_log (
  call_time    TIMESTAMP,
  jira_key     STRING,
  caller       STRING,
  tool         STRING,
```

```
  origin       STRING,      -- 'model' | 'app' | 'job' | 'slack'
  cache_hit    BOOLEAN,
  latency_ms   INT,
  est_cost_usd DOUBLE,
  decision     STRING,      -- 'allow' | 'ask' | 'deny', with the deciding
tier
  reason       STRING
);

-- Derived daily: the motion ledger, grain = jira_key x day
CREATE TABLE gtm_ledger.motion_ledger (
  jira_key              STRING,
  ledger_date           DATE,
  motion_type           STRING,
  owner                 STRING,
  epic_key              STRING,
  serves_priority       STRING,
  canonical_campaign_id STRING,
  sf_campaign_id        STRING,
  ai_tokens             BIGINT,
  ai_cost_usd           DOUBLE,
  zoominfo_cost_usd     DOUBLE,
  labor_cost_usd        DOUBLE,
  dbx_cost_usd          DOUBLE,     -- null until attribution is solved
(section 10)
  motion_cost_usd       DOUBLE,
  cost_avoided_usd      DOUBLE,
  forecast_cost_usd     DOUBLE,     -- from the plan block (section 4.5)
  goal                  STRING,
  goal_attainment       DOUBLE,
  influenced_pipeline_est DOUBLE,   -- always labeled an estimate
  key_validated         BOOLEAN     -- false = flagged, excluded from totals
);
```

`gold.motion_ledger_by_campaign` is the serving view for the dashboards, carrying `owner_user_id`, `territory_id` and `refreshed_at` like every gold table.

8.3 The daily ledger build

1. **Extract usage:** read the gateway usage table for the period; pull `jira_key` from the request tags; price tokens by class (input, output, cache read, cache writes) from the rate card.

2. **Aggregate** AI cost per `jira_key` and day; book untagged usage to an explicit `UNTAGGED` row so it is visible, never dropped.
3. **Join the run log** for ZoomInfo credits, records and human hours; price them from the rate card.
4. **Join the ticket** for owner, motion type, epic and the linked campaign. This requires Jira ticket data in Databricks; the extraction path is confirmed in discovery.
5. **Join the crosswalk and Salesforce** for the canonical campaign, its owner, budget, dates and goal, and the opportunities it influences.
6. **Validate:** every key must match a real Jira ticket and Salesforce record; unmatched keys are flagged and excluded from totals.
7. **Compute** motion cost, cost avoided, goal attainment, influenced pipeline (labeled an estimate under the agreed credit rule), and forecast versus actual.
8. **Publish** the gold view and refresh the motion snapshot the SessionStart hook reads for budget checks.

8.4 Metrics catalog

Metric	Definition	Source	Used for
Motion cost	AI + ZoomInfo + labor (+ Databricks compute when attributable) per motion	Motion ledger	Funding decisions
Cost avoided	Credits not spent because records matched internally; cache hits credited to the asker	Run log, cache hit log	The "spend less by knowing more" case
Goal attainment	Result against the campaign's pre-set goal	Salesforce campaign	Defensible return
Influenced pipeline (estimate)	Opportunities tied to the campaign under the agreed credit rule	Salesforce, Marketo Measure	Directional return, always labeled
Untagged spend share	Untagged AI cost divided by total AI cost	Motion ledger	Health of the stamp; target trends to zero

Metric	Definition	Source	Used for
Forecast versus actual	Plan-block estimate against ledger actual per task	Motion ledger	Whether preloading delivers predictable cost
Cache hit rate and false hits	Hits over lookups; audited wrong answers	Cache hit log	Cache phase exit gate
Calls denied by tier and reason	Count of `deny` and `ask` decisions	Tool-call log	Budgets too tight, roles missing a tool, false blocks
Event-list resolution rate and time to resolve	Resolved records over records in; elapsed time	Run log, holding queue	Value of the proof motion
False-match rate	Wrong matches found in human review	Review outcomes	Tunes the confidence threshold
Intent digest to rep action rate	Digest items followed by a logged rep action	Salesforce tasks and activity	Whether signals are useful
ICP backtest lift	Win rate of high-fit versus low-fit accounts	Databricks	Whether the ICP is right
Cost per resolved record; per qualified meeting	Motion cost over outcomes	Ledger over outcomes	The ROI line for leadership
Dashboard adoption	Weekly active users by role; tool calls per user	Event log	Pilot success
Gold freshness	Tables meeting their `refreshed_at` target	Gold tables	Trust in the dashboards

Spend by user, role, skill and model remains available as a cut of every row above, not as the organizing principle.

9. Roadmap: fund the proof, not the transformation

The proof is about two weeks away; the full vision is a multi-quarter effort whose hardest part is adoption, not engineering. That gap is the argument for starting small: judge the idea on the short distance first, and commit to the long one only once the proof is real. Every phase ends at a gate, and enforcement arrives only after the logs show it will not block legitimate work.

Prove one motion in weeks, then earn each layer at a gate

Phase 0 · Discovery · days
Pull one real gateway usage row; confirm the variant (header tags or usage_context)
Get read access; test one tagged call; map what routes around the gateway
Create the proof's ticket, campaign and crosswalk row; stage the ugliest real list

Gate: one tagged request visible in the usage table

Phase 1 · One-motion proof · about two weeks
Run one real trade-show list through the governed pipeline, writing the run log
Soul and project CLAUDE.md by hand; every call tagged with the ticket
Ledger query for that campaign; fill the manual-versus-governed table with real figures

Gate: leadership sees real cost and return, and decides

Phase 2 · Instrument and harden · weeks
Plugin v0: bootstrap, enrich-list skill, ticket hooks on write tools, logging hooks
Daily ledger build; untagged share tracked; account briefing with a two-call plan
Plan guard and motion budget gate in warn mode; ten briefings, forecast versus actual

Gate: baseline cost per motion known; untagged share falling; forecasts within [__]%

Phase 3 · Role surfaces, MOps first · weeks per role
Gold tables, user_access mapping, server skeleton with OAuth, evals in CI
MOps pilot with the data-quality digest and ledger view, then BDR, then AE and briefs
Each step gated: reconciliation, weekly use, row-scoping tests, security sign-off

Gate: MOps numbers reconcile; security and compliance sign-off for broad rollout

Phase 4 · Govern and scale · quarters
Hooks move from warn to block; per-person caps retired as motion budgets take over
Plugin to SDR, AE and leadership; intent digest, ICP backtest, NBA trigger, org connector
Shared cache only if the ledger shows near-duplicate spend; then promote it to the gateway
Steady state: monthly weight recalibration, quarterly ICP revision, plugin releases like code

roadmap · 5 phases, 4 gates; durations from the source plans

Only Phase 1 is funded now. Each later phase starts when its gate is met, and the role-dashboard and harness rollouts keep their own internal gates inside Phases 3 and 4.

Parked until the ledger earns them: a governed org-wide account table, the full local knowledge wiki beyond the soul, project and campaign nodes, org-wide telemetry dashboards, and the shared semantic cache. Caching belongs to the platform team, and only if the ledger shows a meaningful share of spend on near-duplicate prompts.

Targets in brackets (`[__]`, `[N]`) throughout this document are set from Phase 2 telemetry. We do not guess them up front.

First actions

Everything downstream depends on one row: pull a real request from the gateway's usage or inference table before building anything.

- [ ] Find the gateway's usage table (`system.ai_gateway.usage` or `system.serving.endpoint_usage`) or its inference table in Unity Catalog, and pull one real row
- [ ] From that row, confirm logging is on, which tag field exists (`request_tags` or `usage_context`), and whether payloads are intact or null from the 1 MiB cap
- [ ] Test one tagged call: set `ANTHROPIC_CUSTOM_HEADERS` in a project's settings and check the tag lands in the table
- [ ] Map what routes around the gateway besides the Atlassian MCP
- [ ] Stage the ugliest real trade-show list as a Databricks table; confirm the account and contact tables are queryable
- [ ] Create the Jira ticket and the Salesforce campaign (owner, budget, dates, goal) for the proof, with its crosswalk row
- [ ] Write the soul `CLAUDE.md` and the proof's project `CLAUDE.md` by hand, under 200 lines each
- [ ] Build the pipeline for that one list, writing the motion run log as it goes
- [ ] Write the ledger query for that campaign and fill the before-and-after table with real figures
- [ ] Walk the sponsor through the example privately before the meeting

10. Risks, governance and open decisions

The largest risk is trust: if the first numbers disagree with existing reports, or the first hook blocks legitimate work, adoption stalls. That is why the plan proves one motion before building surfaces, ships MOps first, and starts every gate in warn mode.

10.1 Risks and mitigations

Risk	Effect	Mitigation
Untagged or mis-tagged spend	Ledger undercounts motions; untethered spend looks larger than it is	Tag as a setting, not a prompt; ticket hooks on write tools; untagged spend booked visibly and tracked as a metric; every key validated against Jira and Salesforce
Work routes around the gateway	Invisible spend; a false picture of low usage	Map bypass paths in discovery; declare the measurement boundary; read low usage as a question
Revenue numbers challenged	Credibility of the whole framework	Lead with goal attainment; label influenced pipeline an estimate every time; agree the credit rule before reporting
Duplicates and unresolved lifecycle data	Dashboards and ICP backtest look wrong	Ship surfaces after the audit's identity and lifecycle fixes; show data-quality flags; sequence ICP after the universal person key
Data scope leaks across owners	Compliance incident	Identity-based scoping, Unity Catalog row filters, negative tests in CI; shared wiki layer permission-free by construction; `access_scope` in the cache key
False blocks from hooks	People disable the harness	Warn mode first; block only with evidence; fast override path; deny reasons name the deciding tier

Risk	Effect	Mitigation
A bad hook ships	Everyone's workflow breaks at once	Hooks ship only after the motion is proven by hand; versioned plugin; backward compatible across one minor version
Hooks approximate cost; the gateway is the truth	Users hit gateway limits the harness did not predict	Monthly recalibration of tool weights from gateway data
Hooks in user settings can be edited	User-tier controls are advisory	Enforced hooks in managed settings (`allowManagedHooksOnly`); the gateway as the hard ceiling
Preload bloat outspends the calls it saves	Higher cost per motion	Scoped subgraphs; byte-stable prefix within a session; track input tokens per session; compare forecast with actual
Stale or wrong wiki content	Confidently wrong answers	Named owners per domain, provenance tags, freshness flags, validated-versus-conjecture flags
Teams fork `CLAUDE.md` and drift	Alignment erodes	Bootstrap merges, never clobbers; lineage on local nodes; quarterly refactor
False cache hits	A wrong number in someone's report	Precision-first resolver, exact parameters, logged hits, re-run button; the cache is built only on evidence
Genie billing assumption wrong	Unplanned Databricks spend	Confirm with the billing owner; deterministic SQL or UC functions on the production path; service principals assumed billed
Slack app built on a service account	Budgets and data access collapse to one identity	Per-user mapping from Slack ID to gateway identity
ZoomInfo match errors	Wrong contacts written into the CRM	Confidence gate, holding queue, human review sampling, never fabricate

Risk	Effect	Mitigation
ZoomInfo credit burn	Unplanned spend	Check internal first; enrich known records only; cache results; weekly cadence
Marketo API quota exhausted by extracts	Marketo flows and sync degrade	Bulk extract only, a scheduled window, a quota monitor; gateway cache for repeat requests
Event lists keep bypassing intake	Duplicates and lifecycle-blind records	Marketo-first intake plus a catch flow that stamps unstamped records
Sync latency between Salesforce and Marketo	Gating or intent on stale data	Timestamp every computed field; do not gate on stale opportunity data
Prompt injection through CRM free text	Wrong or unsafe agent actions	Escape text in views; treat tool output as data; no write tools in the dashboard v1
MCP Apps support differs by surface	Some users see only text	Every tool returns a useful text fallback

10.2 Open decisions

Decision	Options	Recommendation	Needed by
Influenced versus sourced credit	Each team gets credit; or one rule decides	Agree one rule with marketing and sales leadership; the ledger supports either	Before any revenue number is shown
Header tags or body `usage_context`	Depends on the gateway variant	Whichever the variant supports; confirm in discovery	Phase 0
Attributing Databricks compute to a motion	Record statement or job IDs in the run log; dedicated warehouses per motion class; leave as overhead	Start by recording IDs in the run log; book the rest as platform overhead	Phase 2

Decision	Options	Recommendation	Needed by
Jira data in Databricks	A connector, an export job, or an API pull in the ledger build	Whichever the platform team already runs	Phase 1
Team tier: restrict only, or also grant	Restrict only; restrict plus a reviewed allowlist	Restrict by default with a small reviewed allowlist for role-essential tools	Phase 2
Role source of truth	Salesforce profile and role; IdP groups	One system, with the policy file generated from it; users without a role get read-only	Phase 2
Shared budget store	Redis, a Databricks table, gateway-native budgets	A Databricks table fed by the ledger, so budgets and actuals share one source	Phase 4
Hook failure mode per tool class	Fail closed or open	Closed for writes, open for reads, failures logged	Phase 2
Starting confidence threshold for writing enriched contacts	Any value	Start at 0.85 and tune from review outcomes	Phase 1
Holding queue location	Databricks only; a Salesforce custom object reps can see	Databricks; a custom object only if reps must see it; never blank-email leads	Phase 1
Crosswalk shape	Custom Metadata Type; custom object	Decide by how often marketing ops creates campaigns	Phase 1
Wiki layout	Set `CLAUDE_CODE_ADDITIONAL_DIRECTORIES_CLAUDE_MD=1`; place the wiki inside each project	Bootstrap decides per machine	Phase 2

Decision	Options	Recommendation	Needed by
Distribution path	Plugin marketplace; Unity Gateway coding-agent configuration	Plugin for skills and hooks; gateway for model routing and MCP services	Phase 2
Dashboard traffic through the gateway	Required by policy or not	Route app tool calls through the gateway as an MCP Service	Phase 3
Dashboard host and on-call	Databricks Apps; internal platform	Decide with the platform team	Phase 3
ZoomInfo display rights	All roles; some roles	Confirm the license before the BDR pilot	Phase 3
Territory source of truth	Salesforce territory management; a separate table	Confirm with sales ops	Phase 3
MOps reconciliation tolerance	Any value	Agree with MOps before the first gate	Phase 3
Dashboard write actions (v2)	When, and under what approval model	After the AE phase gate, with a user-in-the-loop approval model	After Phase 3

10.3 Questions for the platform owners

Phrase these as discovery, never accusation: "I think the data's already being captured; can you help me find it?"

Gateway owner

1. Which gateway variant do we run, and is usage tracking or inference-table logging enabled on our endpoint?
2. Can we get read access, or a filtered view, on the usage table? It is admin-only by default.
3. Can we expose per-user and per-tool consumption, not just per-model?
4. What else routes around the gateway?

5. Is Claude Code routed through Unity Gateway with `ug` or manual settings, and is the 1-hour prompt cache enabled? Which model services and budgets apply to GTM users?

Databricks and Salesforce owners

1. Will Genie calls from Claude Code run as each user (free through January 31, 2027) or through a service principal (billed)?
2. Are the MCP Services we need configured per user (inherits access) or with a shared principal (does not)?
3. Is Salesforce Hosted MCP enabled, and do its calls count against the daily API allocation?
4. Is a Lakeflow Connect pipeline landing Campaign, CampaignMember, Lead, Contact, Account and Opportunity, and on what schedule?
5. Does the crosswalk live as Custom Metadata or a custom object, given how often campaigns are created?

11. Appendices

A. Leadership delivery kit

Lead with the reframe, prove it with one campaign, gesture once at the horizon, and stop. Slides 2 to 5 carry the weight; restraint on slide 6 is what reads as leadership.

Elevator pitch (60 seconds). We're about to cap AI spend per person to control cost. But capping people doesn't control cost; it rations innovation, and our best users hit the limit first. I think we flip it: measure AI by what the spending produces, not who spends it. Take one real workflow, turning a trade-show list into a running campaign, and rebuild it so it checks what we already own before we pay to enrich, loads clean, and ties every token to a campaign through its Jira ticket. Do that, and cost control and ROI stop being two separate worries. They're the same number. I've built one example on a real campaign; happy to walk anyone through it.

#	Slide title	The one point it makes	Time
1	Who's spending vs. what it's producing	Caps ration innovation; role views fall out free from a motion model	2 min
2	One campaign, the way it works today	Ragged list, blind ZoomInfo spend, hand-loading, attribution lost	2 min

#	Slide title	The one point it makes	Time
3	The same campaign, governed	Check what we own, enrich the gap, load clean, tie to a campaign	3 min
4	The cost AI removes, not the cost AI adds	Before and after: hours saved, enrichment avoided, tokens now visible	3 min
5	Where cost and return finally meet	Campaign with owner, budget, dates, goal; three honest numbers	2 min
6	One motion is the pattern	Ticket to campaign to opportunity to account; gesture only, then sit down	30 sec

Talking points. Three themes: we're measuring the wrong thing; spend less by knowing more; prove return by tying work to outcome.

- On control: "We're not removing control. We're upgrading it from a blunt cap to a targeted one."
- On transition: "Keep a loose guardrail now. As motion visibility comes online, hard caps become unnecessary."
- On the data: "The gateway is almost certainly logging what we need already. Nobody has built the per-motion view yet." Never imply the data is being hidden.
- On honesty: label influenced pipeline as an estimate, every time. It buys credibility for the numbers that are hard.
- On low usage: "Little AI on a motion is a question, not a virtue: it's either manual work or work we can't see."

Framing rules. Present it as how we get what the current plan is trying to achieve, never as "your plan is wrong"; the audience should feel rescued, not corrected. Pre-wire: walk the sponsor through the example privately first, offered as support for her cost-and-ROI story. Bring others in afterward, giving owners of adjacent systems a visible win on their own turf. Show one undeniable example, mention the larger pattern once, then stop.

Anticipated questions. Every hard question has a short answer that points back to the one-motion proof.

Question	Answer
So we just let spend run wild?	No. We replace a blind control with a smart one. Visibility lets us cut spend that isn't producing instead of throttling everyone equally.
Where do we start?	One motion, not a platform: the trade-show list. Confirm the data exists (days), build the motion (about two weeks), bring back real numbers.
Where does the measurement take place?	In a derived Databricks table that joins gateway usage to business objects through the Jira key. Salesforce and Jira stay the sources of truth, joined at query time.
Where are we now vs. where we need to be?	Raw materials, no assembly. We see spend per model only. The proof is two weeks away; org-wide adoption is quarters, and adoption is the hard part.
What do you need from us?	Read access to the gateway usage logs, and a nod to build one example before committing to role-based dashboards.
Why not build the role dashboards first?	Role views fall out of the motion model for free: every ticket has an owner. Built the other way, motion views are impossible.
What if the model mangles the IDs?	It never sees them. The ticket key travels as request metadata, not prompt text, and every key is validated against real records.
Isn't this a lot of engineering?	The proof is scripts on one machine. The plugin, hooks and skills library come only after the proof earns them.
How do we get people to adopt it?	Make the governed path the easy path: one command beats building a spreadsheet. Enforcement rides in hooks so nobody has to remember the rules.
What about work outside the gateway?	It's invisible to this telemetry, and we'll say so. Mapping what routes around the gateway is part of discovery.

Question	Answer
Doesn't low AI usage mean efficiency?	Not necessarily. It may mean manual work, usually the most expensive way, or a tool we can't see. It marks where to look next.
How confident are the revenue numbers?	Goal attainment is defensible. Influenced pipeline is labeled an estimate, and the credit rule gets agreed before we report it.

B. Unified plugin layout

One plugin replaces the three layouts in the source designs. Plugins cannot ship `CLAUDE.md` files, so templates are written by the bootstrap skill. Skill and agent names are proposed.

```
gtm-plugin/
  .claude-plugin/plugin.json        # name, semver, description
  hooks/hooks.json                  # the inventory in section 5.4
  scripts/
    session_start.py                # context, JIRA_KEY export, motion budget
snapshot
    ticket_gate.py                  # hooks 1 and 3
    check_internal.py               # hook 5
    budget_gate.py                  # hook 6: plan guard, role tools, motion
budget
    ledger.py  write_back.py        # hooks 7 and 8
    subagent_stop.py  stop.py       # hooks 9 and 10
  policy/
    gtm-policy.json                 # generated from the role source of truth
    policy.yaml                     # budgets, tiers, mode (warn | block)
  skills/
    bootstrap/  enrich-list/  account-brief/  report-query/  intent-digest/
    icp-score/  nba/  cost-estimate/  wiki-lookup/  wiki-update/
    quarterly-refactor/  promotion/  provenance/  freshness/  cache-admin/
  agents/
    enrichment-matcher.md           # small model, read-only tools
    match-resolver.md  icp-scorer.md
    retriever.md  formatter.md      # cheap tier
    analyst.md                      # standard tier
  commands/
    enrich-list.md  brief.md  report.md  cost.md  refresh-wiki.md
```

```
  templates/
    CLAUDE.root.md                  # soul tier: standing motion rules
    project/CLAUDE.md               # motion + ticket
    project/.claude/settings.json   # ANTHROPIC_CUSTOM_HEADERS tag
  .mcp.json                         # Marketo, Salesforce, Databricks, Genie,
ZoomInfo via the gateway
```

Role matrix (starting point; budgets set from the first month of ledger data):

Role	Enabled skills	MCP access	Default model
SDR / BDR	Account briefing, intent digest	Salesforce read, ZoomInfo read, Slack post	Small model for lookups
AE	Account briefing, opportunity context	Salesforce read and write on own records, ZoomInfo read	Standard
Marketing ops	Event-list resolve, intent digest, ICP score	Marketo read and write, Salesforce read and write, ZoomInfo read	Standard
GTM engineering	All	All, including admin tools	Standard; large on demand
Leadership	Briefings, reports	Read only	Standard

Install and lifecycle. Bootstrap is idempotent: it detects existing `CLAUDE.md` files and merges, never clobbers, and creates root and project tiers, wiki folders and `policy.yaml` for users starting from zero. A team git repo hosts an internal marketplace (`.claude-plugin/marketplace.json`); org-managed settings can pre-register it, force-install the plugin and pin the gateway. Semver on the plugin, a version key in `policy.yaml` , migrations for the cache schema; hooks stay backward compatible across one minor version. Unity Gateway can also publish models, MCP servers and skills under Govern, Coding Agents (Enhanced Unity Gateway preview; gateway skills in Beta); the plugin path is the one that carries hooks.

C. Context architecture decision log

Twenty-seven decisions shaped the context design; three were revised along the way (13, 20, 23) and one (26) needs revision after billing validation.

#	Decision	Status
1	Preload context locally before any gateway call	Decided
2	A soul `CLAUDE.md` above all project folders; each project inherits and specializes	Decided, validated
3	Must-happen work lives in skills and code; judgment lives in `CLAUDE.md`	Decided, validated
4	Everyone shares a global base; each person grows a private local overlay	Decided
5	Quarterly global updates applied by a person-invoked refactor skill	Decided
6	Cadence, mutability and cascade are properties of each node or layer	Decided
7	Knowledge splits into world, market and local layers	Decided
8	The campaign node (Salesforce campaign = Marketo program) is the keystone	Decided, keystone
9	Org layer and GTM layer are distinct, connected graphs	Decided
10	A validated account is a persistent node refreshed daily or hourly	Decided
11	Two clocks: events refresh facts; the fiscal calendar re-colors interpretation	Decided
12	Databricks is the internal CDP fed by every touchpoint, with validated crosswalks	Decided
13	Marketo is the single front door; data flows Marketo first, then native sync	Revised, decided
14	The crosswalk is a metadata table in Salesforce with IDs, lead source, NBA and the attributable-event definition	Decided
15	NBA fires on an attributable last touch, read from campaign membership	Decided

#	Decision	Status
16	Every campaign and signal is measured against the ICP; qualification tests the ICP	Decided
17	Route by data type across wiki, Genie and gateway	Decided
18	Shared layers may centralize behind the gateway; the personal overlay stays local	Proposed
19	Anything that improves first-shot quality is preloaded locally	Decided
20	Interpretive context preloads; atomic facts come live	Revised into 21
21	A grid: stable interpretive (soul), moving interpretive (market), atomic live (gateway)	Decided
22	Preloaded markdown carries a plan of pre-declared tool calls, each mapped to a skill	Decided
23	Executive goals are an external anchor the wiki points to, linked down to Jira epics	Revised, decided
24	The wiki stores query intent and interpretation, not live Salesforce rows	Decided
25	Aggregates go to Databricks; live single-record lookups go to Salesforce	Decided
26	Genie runs as a scheduled batch engine on a separate budget	Needs revision (billing)
27	Live and local paths self-gate by OAuth; the shared layer is permission-free by construction	Decided

This plan adds one decision above all of them: **the motion, declared as a Jira ticket, is the unit every layer is measured by.**

D. Validation summary

Of 20 technical claims checked against documentation in the context architecture work, 12 were confirmed, 5 confirmed with a correction, 2 needed a correction and 1 is unverified. The corrections that change this plan:

1. There is no single inference call once tools run: Claude Code sends a request per tool-loop step, re-sending the context, so prompt caching is part of the cost model.
2. The assembled prompt crosses the gateway; reading local files does not.
3. Genie is not simply a free separate budget: free for people until January 31, 2027; service principals billed from the first DBU; warehouse compute always billed.
4. Genie MCP tool names (`genie_ask` , `genie_poll_response`) came from a community post and are unverified; discover them with `tools/list` .
5. SessionStart hook context is capped at 10,000 characters per field; preloaded subgraphs must be compact.
6. Names changed: Unity Gateway (Unity AI Gateway) and Genie Agents.
7. Tagging is a setting, not a hook: `ANTHROPIC_CUSTOM_HEADERS` , not a hook, stamps model requests (from the motion framework).
8. MCP Apps protocol names come from a secondary write-up of spec 2026-01-26 and must be confirmed against the official text (from the lead-card design).

E. Glossary

Term	Meaning in this plan
Motion	A repeatable GTM workflow with a business object attached, declared as a Jira ticket
Motion ledger	The derived Databricks table joining AI usage, run logs and business objects by `jira_key`
Motion run log	Per-run record of records in, matched, enriched, credits, loaded and human hours
Platform ticket	A reserved ticket for shared infrastructure spend, reported as overhead
Cost avoided	Spend not incurred because data was already owned or an answer was cached
Canonical campaign ID	The system-independent campaign key in the crosswalk
Campaign crosswalk	One governed row per campaign: IDs, lead source, desired action, attributable event, NBA, priority

Term	Meaning in this plan
Attributable event	A campaign member status flagged as Responded (`HasResponded = true`)
NBA	Next-best-action: fired on an attributable last touch, informed by full account history
Soul	The top-tier `CLAUDE.md` above all project folders
Local overlay	A person's private context, which never leaves the machine unless promoted
Plan block	Pre-declared tool calls per task, each with a condition and budget
Unity Gateway	Databricks' AI gateway (formerly AI Gateway)
Genie Agent	A curated Genie space (formerly Genie space)
MCP App / View	A tool-bound `ui://` HTML resource rendered by the host in a sandboxed iframe
Gold table	A Databricks serving table at the grain a dashboard view shows
Harness	The local scaffolding (tiers, hooks, skills, cache) that keeps Claude Code on a governed path

F. Sources

This document consolidates six source documents. Technical claims carry the validation status those documents recorded as of their dates; they were not re-verified for this consolidation.

Document	Date	Contributes
Measure AI by Motion, Not by Head: GTM AI Cost and ROI Framework	Oct 12, 2026	The thesis, ledger, ROI model, proof motion, hooks, roadmap, delivery kit
GTM Context Architecture: Plan of Record	Oct 5, 2026	`CLAUDE.md` tiers, knowledge layers, routing, cost model, crosswalk, NBA, Genie and Salesforce paths, decision log, validation ledger

Document	Date	Contributes
The Governed AI Harness (deck)	October 2026	Harness components, declarations, hook lifecycle, tiered cache, resolver, cache schema, rollout
Governed Claude Code for GTM Engineering	Oct 12, 2026	Three-tier governance, precedence, budget gate, role plugin, Slack surface, workflows, data foundations, metrics
GTM Role Dashboards: MCP App Technical Design	Oct 12, 2026	Data layers, MCP server and tools, identity, role specs, headless jobs, deployment, rollout
Lead Card Call Path (interactive)	2026	MCP Apps call sequence, CSP rule, `data-*` attributes, event log

External references cited by those documents: Databricks (Unity AI Gateway usage tracking; Configure AI Gateway on model serving endpoints; Connect Claude Code to Unity Gateway; Unity Gateway release notes; managed MCP servers; Genie Agent MCP server; Genie Agents API; MCP Services; Monitor AI Gateway usage; Genie cost, budgets and pricing; Salesforce ingestion connector limitations), Anthropic (Claude Code memory and `CLAUDE.md`; hooks reference; environment variables; plugins overview; prompt caching), Salesforce and Marketo (Hosted MCP Servers; API request limits; CampaignMember reference; Custom Metadata Types; Marketo campaign sync; community threads on duplicates and sync timing), and the MCP Apps extension (modelcontextprotocol/ext-apps).